Question Bank for FA-1 (Unit 1 and Unit 2)

1. Explain how you would detect a cycle in a singly linked list. Write the algorithm for cycle detection.

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* detectCycle(ListNode* head) {
    if (head == nullptr || head->next == nullptr) {
        return nullptr;  // No cycle if the list is empty or has only one node
    }

    // Initialize slow and fast pointers
    ListNode* slow = head;
    ListNode* fast = head;

    // Detect cycle using Floyd's cycle detection algorithm
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;        // Slow pointer moves one step
        fast = fast->next->next;  // Fast pointer moves two steps

        // If cycle is detected
        if (slow == fast) {
            // Reset one pointer to head and move both pointers at the same speed
            slow = head;
            while (slow != fast) {
                slow = slow->next;
                fast = fast->next;
            }
            // The meeting point is the start of the cycle
            return slow;
        }
    }

    // No cycle found
    return nullptr;
}
```

To detect a cycle in a singly linked list, you can use **Floyd's Cycle Detection Algorithm** (also known as the **Tortoise and Hare Algorithm**). This algorithm uses two pointers that traverse the linked list at different speeds. If there is a cycle in the list, the two pointers will eventually meet; if there is no cycle, the faster pointer will reach the end of the list.

How Floyd's Cycle Detection Algorithm Works:

1. **Initialize Two Pointers**:
    - A slow pointer (slow) that moves one step at a time.
    - A fast pointer (fast) that moves two steps at a time.
2. **Traverse the List**:
    - Move the slow pointer by one node and the fast pointer by two nodes.
    - If there is no cycle, the fast pointer will reach the end of the list (null or None), indicating that the list does not have a cycle.

    - If there is a cycle, the slow and fast pointers will eventually meet inside the cycle.
3. **Detect the Cycle**:
    - If the slow and fast pointers meet at some point, it means the list contains a cycle.

Main Logic:

1. Initialization: Start with two pointers:
  - `slow` starts at the head and moves one step at a time.
  - `fast` starts at the head but moves two steps at a time.

2. Cycle Detection: The `slow` pointer moves one node at a time, while the `fast` pointer moves two nodes at a time. If there is a cycle, these two pointers will eventually meet within the cycle. If no cycle exists, `fast` will reach the end (`nullptr`).

3. Start of Cycle Detection: Once a cycle is detected, reset the `slow` pointer to the head of the list. Now move both `slow` and `fast` pointers one step at a time. The point at which they meet again is the start of the cycle.

Algorithm Steps:

1. Step 1 - Initialization:
  - Start by initializing two pointers: `slow` and `fast`, both pointing to the head of the linked list.
doc
2. Step 2 - Cycle Detection:
  - Move `slow` by one step and `fast` by two steps in each iteration.
  - Continue this process until:
    - `fast` or `fast->next` becomes `nullptr` (this indicates there is no cycle).
    - Or, `slow` and `fast` meet (this indicates a cycle).

3. Step 3 - Finding the Start of the Cycle:
  - If a cycle is detected (i.e., `slow == fast`):
    - Reset the `slow` pointer to the head of the list.
    - Move both `slow` and `fast` one step at a time until they meet again.
    - The node where they meet is the starting node of the cycle.

4. Step 4 - Return the Result:
  - If a cycle is found, return the node where `slow` and `fast` meet after resetting `slow` to the head.
  - If no cycle is found, return `nullptr`.

Time Complexity:
-O(n): Both the cycle detection and finding the start of the cycle take linear time relative to the number of nodes in the list.
Space Complexity:
- O(1): The algorithm uses only a constant amount of extra space for the two pointers (`slow` and `fast`).
This method is efficient and gives you more information by returning the starting node of the cycle, rather than just confirming if a cycle exists.

2. Describe how you would remove duplicate elements from a singly linked list.

Outer Loop: Traverse each node using the pointer curr1.
Inner Loop: For each node pointed to by curr1, use curr2 to check for duplicates in the remaining part of the list.
Duplicate Removal: If a duplicate is found, remove it by skipping the duplicate node and adjusting the pointers.
Memory Deallocation: After removing the duplicate, free the memory associated with the removed node.

```
#include <iostream>
#include <unordered_set>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) {
        data = val;
        next = nullptr;
    }
};

// Function to remove duplicates using nested loops
Node* removeDuplicates(Node* head) {
    Node* curr1 = head;

    // Traverse each node in the list
    while (curr1 != nullptr) {
        Node* curr2 = curr1;

        // Traverse the remaining nodes to find and remove duplicates
        while (curr2->next != nullptr) {

            // Check if the next node has the same data as the current node
            if (curr2->next->data == curr1->data) {

                // Duplicate found, remove it
                Node* duplicate = curr2->next;
                curr2->next = curr2->next->next;

                // Free the memory of the duplicate node
                delete duplicate;
            } else {

                // If the next node has different data from
                // the current node, move to the next node
                curr2 = curr2->next;
            }
        }

        // Move to the next node in the list
        curr1 = curr1->next;
    }
    return head;
}

void printList(Node* head) {
    Node* curr = head;
    while (curr != nullptr) {
        cout << curr->data << " ";
        curr = curr->next;
    }
    cout << endl;
```

```
}

// Function to append a new node to the end of the list
void append(Node*& head, int newData) {
    Node* newNode = new Node(newData);
    if (head == nullptr) {
        head = newNode;
        return;
    }

    Node* last = head;
    while (last->next != nullptr) {
        last = last->next;
    }
    last->next = newNode;
}

int main() {
    Node* head = nullptr;
    int n;

    // Prompt user for input
    cout << "Enter the number of elements in the linked list: ";
    cin >> n;

    // Taking user input for list nodes
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; ++i) {
        int val;
        cin >> val;
        append(head, val); // Append the value to the linked list
    }

    cout << "Original list: ";
    printList(head);

    // Remove duplicates from the linked list
    head = removeDuplicates(head);

    cout << "List after removing duplicates: ";
    printList(head);

    return 0;
}
```

Main Logic for Removing Duplicates in an Unsorted Singly Linked List.The approach used in the code is based on a nested loop strategy, where we compare each node with every subsequent node to check for duplicates and remove them.

Here's the step-by-step breakdown of the main logic:

Step-by-Step Explanation:

1. Outer Loop (Traversal of the List):
    - We traverse each node in the list using a pointer curr1.
    - For each node that curr1 points to, we compare it with all subsequent nodes in the list to detect duplicates.
2. Inner Loop (Checking for Duplicates):
    - For each node pointed by curr1, we set another pointer curr2 to the same node.
    - The inner loop (curr2->next) checks every node after curr1 for a duplicate value.
    - If a duplicate node is found (curr2->next->data == curr1->data):
        - The duplicate node (curr2->next) is skipped by adjusting the next pointer of curr2 to point to the node after the duplicate (curr2->next = curr2->next->next).
        - The memory of the duplicate node is then freed using delete to prevent memory leaks.
    - If no duplicate is found, curr2 moves to the next node.
3. Move to Next Node:
    - After completing the inner loop, we move curr1 to the next node and repeat the process until we have checked all nodes in the list.

Time Complexity:

- The time complexity is $O(n^2)$ because for each node, the inner loop iterates through the remaining nodes to find duplicates. Thus, we have a nested loop structure where each node is compared with every other node in the list.

3. Write a function to partition a singly linked list around a value x, such that all nodes less than x come before nodes greater than or equal to x.

Approach:
The idea is to use three dummy nodes to create three separate partitions: less, equal, and greater. As the list is traversed, each node is added to its corresponding partition. Once all nodes are processed, the partitions are connected to form the final list.

```
#include <iostream>
using namespace std;

class Node {
public:
   int data;
   Node* next;

   Node(int val) {
      data = val;
      next = NULL;
   }
};

Node* partition(Node* head, int x) {
   // Dummy head for nodes less than x
   Node* lessHead = new Node(0);

   // Dummy head for nodes equal to x
   Node* equalHead = new Node(0);
```

```
    // Dummy head for nodes greater than or equal to x
    Node* greaterHead = new Node(0);

    Node* less = lessHead;
    Node* equal = equalHead;
    Node* greater = greaterHead;

    Node* curr = head;

    while (curr != NULL) {
        if (curr->data < x) {
            less->next = curr;
            less = less->next;
        } else if (curr->data == x) {
            equal->next = curr;
            equal = equal->next;
        } else {
            greater->next = curr;
            greater = greater->next;
        }
        curr = curr->next;
    }

    // Connect the partitions together
    greater->next = NULL;

    // Connect equal to greater
    equal->next = greaterHead->next;

    // Connect less to equal
    less->next = equalHead->next;

    // New head of the rearranged list
    Node* newHead = lessHead->next;

    // Clean up dummy nodes
    delete lessHead;
    delete equalHead;
    delete greaterHead;

    return newHead;
}

void printList(Node* head) {
    Node* curr = head;
    while (curr != NULL) {
        cout << curr->data << " ";
        curr = curr->next;
    }
    cout << endl;
}

int main() {
    // Creating the linked list: 1 -> 4 -> 3 -> 2 -> 5 -> 2
    Node* head = new Node(1);
    head->next = new Node(4);
```

```
    head->next->next = new Node(3);
    head->next->next->next = new Node(2);
    head->next->next->next->next = new Node(5);
    head->next->next->next->next->next = new Node(2);

      int x = 3;
    head = partition(head, x);
    printList(head);

    return 0;
}
```

1 2 2 3 4 5
Time complexity: O(n)
Auxiliary Space: O(1)

4. You are given an integer array score of size n, where score[i] is the score of the ith athlete in a competition. All the scores are guaranteed to be unique. The athletes are placed based on their scores, where the 1st place athlete has the highest score, the 2nd place athlete has the 2nd highest score, and so on. The placement of each athlete determines their rank: The 1st place athlete's rank is "Gold Medal". The 2nd place athlete's rank is "Silver Medal". The 3rd place athlete's rank is "Bronze Medal". For the 4th place to the nth place athlete, their rank is their placement number (i.e., the xth place athlete's rank is "x").

```
#include <iostream>
#include <string>
using namespace std;

// Function to perform Insertion Sort based on scores in descending order
void insertionSort(int scores[], int indices[], int n) {
    for (int i = 1; i < n; i++) {
        int keyScore = scores[i];
        int keyIndex = indices[i];
        int j = i - 1;

        // Move elements of scores[0..i-1] that are less than keyScore to one position ahead
        while (j >= 0 && scores[j] < keyScore) {
            scores[j + 1] = scores[j];
            indices[j + 1] = indices[j];
            j--;
        }
        scores[j + 1] = keyScore;
        indices[j + 1] = keyIndex;
    }
}

void findRelativeRanks(int score[], string result[], int n) {
    int scores[n], indices[n];

    // Copy scores and track original indices
    for (int i = 0; i < n; i++) {
        scores[i] = score[i];
        indices[i] = i;  // Store the original index
    }

    // Sort scores in descending order using insertion sort
    insertionSort(scores, indices, n);
```

```
    // Assign ranks based on the sorted order
    for (int i = 0; i < n; i++) {
        if (i == 0) {
            result[indices[i]] = "Gold Medal";
        } else if (i == 1) {
            result[indices[i]] = "Silver Medal";
        } else if (i == 2) {
            result[indices[i]] = "Bronze Medal";
        } else {
            result[indices[i]] = to_string(i + 1);
        }
    }
}

int main() {
    int n;

    // Input the number of athletes
    cout << "Enter the number of athletes: ";
    cin >> n;

    int score[n];
    string result[n];

    // Input the scores of athletes
    cout << "Enter the scores of athletes: ";
    for (int i = 0; i < n; i++) {
        cin >> score[i];
    }

    // Call the function to find relative ranks
    findRelativeRanks(score, result, n);

    // Output the ranks
    cout << "Relative Ranks: ";
    for (int i = 0; i < n; i++) {
        cout << result[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Explanation :
1. **Insertion Sort**:
   - The sorting logic has been changed to use Insertion Sort, which iterates over the array and places each element in its correct position by shifting larger elements to the right.

2. **Sorting**:
   - The Insertion Sort is performed on the `scores[]` array in descending order, while maintaining the original indices in `indices[]`.

3. **Rank Assignment**:

- Once sorted, the top three athletes are assigned "Gold Medal", "Silver Medal", and "Bronze Medal". Others are assigned their ranks as their position in the sorted order (i.e., `4`, `5`, etc.).

How Insertion Sort Works:
- For each element, it compares it with previous elements in the sorted portion of the array and shifts them if they are smaller.
- This ensures the array is sorted as new elements are inserted into their correct positions.

5. Given the array [22, 7, 2, 19, 10,18,15,13] and using the gap sequence [3, 1], show the sorting process with Shell Sort.

Certainly! Here's the shell sort process for the given array [22, 7, 2, 19, 10, 18, 15, 13] using the gap sequence [3, 1]:

Gap: 3

1. Pass 1 (i = 0): Compare 22 with 7 (no swap needed).
2. Pass 1 (i = 3): Compare 19 with 2 (swap: 19 becomes 2, 2 becomes 19). Array becomes: [22, 7, 19, 10, 18, 15, 13, 2]
3. Pass 1 (i = 6): Compare 15 with 12 (no swap needed).

Gap: 1

1. Pass 2 (i = 1): Compare 7 with 19 (no swap needed).
2. Pass 2 (i = 2): Compare 19 with 2 (no swap needed).
3. Pass 2 (i = 3): Compare 10 with 19 (no swap needed).
4. Pass 2 (i = 4): Compare 18 with 10 (swap: 18 becomes 10, 10 becomes 18). Array becomes: [22, 7, 19, 18, 10, 15, 13, 2]
5. Pass 2 (i = 5): Compare 15 with 18 (no swap needed).
6. Pass 2 (i = 6): Compare 13 with 15 (no swap needed).
7. Pass 2 (i = 7): Compare 2 with 13 (swap: 2 becomes 13, 13 becomes 2). Array becomes: [22, 7, 19, 18, 10, 15, 2, 13]

Final Sorted Array: [2, 7, 10, 13, 15, 18, 19, 22]

Explanation:

- We start with a gap of 3, which means we compare elements that are 3 positions apart in the array.
- In the first pass with gap 3, we only need one swap (between elements at indices 3 and 6).
- We then reduce the gap to 1, which allows us to compare adjacent elements and perform any necessary swaps.
- In the second pass with gap 1, we perform two swaps to move elements into their correct positions.
- After processing both gaps, the array becomes completely sorted.

6. Write a function for a given array of integers nums, sort the array in increasing order based on the frequency of the values. If multiple values have the same frequency, sort them in decreasing order. Return the sorted array.

a. Eg. Input: [25,7,1,1,5,7,1,10,7]

b. Output: [5,10,25,7,1]
#include <iostream>

```
using namespace std;

// Function to sort the array based on frequency using Insertion Sort
void frequencySort(int nums[], int n) {
    int frequency[101] = {0}; // Frequency array for values (assuming range 0 to 100)
    int unique[101]; // Array to hold unique elements
    int uniqueCount = 0; // Number of unique elements

    // Count frequencies of each element
    for (int i = 0; i < n; i++) {
        frequency[nums[i]]++;
    }

    // Collect unique elements
    for (int i = 0; i < 101; i++) {
        if (frequency[i] > 0) {
            unique[uniqueCount++] = i;
        }
    }

    // Insertion sort based on frequency and value
    for (int i = 1; i < uniqueCount; i++) {
        int key = unique[i];
        int j = i - 1;

        // Sort by frequency, and if frequencies are equal, sort by value
        while (j >= 0 &&
               (frequency[unique[j]] > frequency[key] ||
               (frequency[unique[j]] == frequency[key] && unique[j] < key))) {
            unique[j + 1] = unique[j];
            j--;
        }
        unique[j + 1] = key;
    }

    // Output the sorted array based on frequency
    cout << "Sorted Array: ";
    for (int i = 0; i < uniqueCount; i++) {
        for (int j = 0; j < frequency[unique[i]]; j++) {
            cout << unique[i] << " "; // Print each unique element based on frequency
        }
    }
    cout << endl;
}

int main() {
    int nums[] = {25, 7, 1, 1, 5, 7, 1, 10, 7};
    int n = sizeof(nums) / sizeof(nums[0]);

    frequencySort(nums, n);

    return 0;
}
```

1. **Sorting Logic**:

- If the frequency of the current element is higher, it remains in its place. If two elements have the same frequency, they are sorted in descending order by their value.

2. **Frequency Output**:
   - The sorted unique elements are printed multiple times according to their frequency in the original array.

How Insertion Sort Works:
- It picks each element one by one and places it in the correct position in the sorted part of the array by shifting elements that are greater in frequency or smaller in value.

Example Output:
For the input `{25, 7, 1, 1, 5, 7, 1, 10, 7}`, the sorted output will be:
```
Sorted Array: 25 10 5 7 7 7 1 1 1
```

7. Given two strings a and b consisting of lowercase characters. The task is to check whether two given strings are an anagram of each other or not. An anagram of a string is another string that contains the same characters, only the order of characters can be different. For example, act and tac are an anagram of each other. Strings a and b can only contain lower case alphabets.

```
#include <iostream>
#include <string>

using namespace std;

int areAnagrams(string a, string b) {
    if (a.length() != b.length())
        return 0; // If lengths differ, they are not anagrams

    int freq[26] = {0}; // Array to count the frequency of characters

    for (int i = 0; i < a.length(); ++i) {
        freq[a[i] - 'a']++; // Increment for character in a
        freq[b[i] - 'a']--; // Decrement for character in b
    }

    for (int i = 0; i < 26; ++i) {
        if (freq[i] != 0)
            return 0; // Not anagrams if frequencies don't match
    }

    return 1; // Strings are anagrams
}

int main() {
    string a, b;
    cout << "Enter first string: ";
    cin >> a;
    cout << "Enter second string: ";
    cin >> b;

    if (areAnagrams(a, b))
        cout << a << " and " << b << " are anagrams." << endl;
    else
        cout << a << " and " << b << " are not anagrams." << endl;
```

```
    return 0;
}
```

Main logic:

1. Length Check: If the lengths of the two strings differ, they cannot be anagrams.
2. Frequency Count: A freq array tracks the frequency of characters in both strings:
    - For each character in the first string, the count is incremented.
    - For each character in the second string, the count is decremented.
3. Verification: If all elements in the freq array are zero at the end, the strings have the same characters in equal amounts, and thus are anagrams.

This logic ensures that the two strings are anagrams if the frequencies of all characters match exactly.

8. Given an array of integers nums, sort the array in increasing order based on the frequency of the values. If multiple values have the same frequency, sort them in decreasing order. Return the sorted array.

```
#include <iostream>

using namespace std;

// Function to swap two integers

void swap(int &a, int &b) {

    int temp = a;

    a = b;

    b = temp;

}

// Function to sort the array using Insertion Sort based on frequency and value

void sortByFrequency(int nums[], int freq[], int size) {

    for (int i = 1; i < size; i++) {

        int keyNum = nums[i];

        int keyFreq = freq[i];

        int j = i - 1;

        // Sort by frequency first, then by value (decreasing) if frequencies are equal

        while (j >= 0 && (freq[j] > keyFreq || (freq[j] == keyFreq && nums[j] < keyNum))) {

            nums[j + 1] = nums[j];
```

```
            freq[j + 1] = freq[j];
            j–;   }
        nums[j + 1] = keyNum;
        freq[j + 1] = keyFreq;
    }
}
int main() {
    int nums[] = {1, 1, 2, 2, 2, 3};
    int size = sizeof(nums) / sizeof(nums[0]);
    // Step 1: Create frequency array
    int freq[100] = {0}; // Assuming the numbers are between 0 and 99
    // Step 2: Count frequencies
    for (int i = 0; i < size; ++i) {
        freq[nums[i]]++;
    }
    // Step 3: Create a sorted array using the custom frequency and value rules
    // To keep track of the numbers and their frequencies, we create two arrays
    int sortedNums[size], sortedFreq[size];
    int index = 0;
    // Add each number from nums[] with its frequency to the sorted arrays
    for (int i = 0; i < 100; ++i) {
        if (freq[i] > 0) {
            for (int j = 0; j < size; ++j) {
                if (nums[j] == i) {
                    sortedNums[index] = nums[j];
                    sortedFreq[index] = freq[i];
                    index++;
                }
```

```
            }
        }
    }

    // Step 4: Sort by frequency and value using Insertion Sort
    sortByFrequency(sortedNums, sortedFreq, size);

    // Step 5: Output the sorted array
    cout << "Sorted array: ";
    for (int i = 0; i < size; ++i) {
        cout << sortedNums[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Explanation :

1. **Insertion Sort**:
   - The `sortByFrequency` function now uses **Insertion Sort** to sort the `nums[]` and `freq[]` arrays.
2. **Sorting Criteria**:
   - The sorting is performed first by frequency in ascending order.
   - If two elements have the same frequency, they are sorted in **decreasing order of value**.
3. **Insertion Sort Logic**:
   - Starting from the second element (`i = 1`), each element is compared with the previous elements (`j = i - 1`) and inserted into the correct position based on the sorting rules (frequency first, then value).

Example Output:

For the input `{1, 1, 2, 2, 2, 3}`, the output will be:

Sorted array: 3 1 1 2 2 2

9. Given an integer array nums, return the third distinct maximum number in this array. If the third maximum does not exist, return the maximum number.

```
#include <iostream>
using namespace std;

// Function to perform insertion sort
void insertionSort(int arr[], int size) {
    for (int i = 1; i < size; i++) {
        int key = arr[i];
        int j = i - 1;

        // Move elements of arr[0..i-1], that are greater than key, to one position ahead
        // of their current position
        while (j >= 0 && arr[j] < key) {  // Sort in descending order
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

// Function to find the third distinct maximum
int findThirdDistinctMax(int arr[], int size) {
    // First, sort the array in descending order
    insertionSort(arr, size);

    int distinctCount = 1;  // Start with 1 because the first element is the largest
    int thirdMax = arr[0];  // Initialize thirdMax as the first element

    // Iterate through the sorted array to find the third distinct maximum
    for (int i = 1; i < size; i++) {
        if (arr[i] != arr[i - 1]) {  // If the current element is distinct from the previous one
            distinctCount++;
        }
        if (distinctCount == 3) {  // When the third distinct element is found
            thirdMax = arr[i];
            break;
        }
    }

    // If less than 3 distinct elements are found, return the largest element
    if (distinctCount < 3) {
        return arr[0];  // Return the largest element if no third distinct maximum
    }

    return thirdMax;  // Return the third distinct maximum
}

int main() {
    int nums[] = {2, 2, 3, 1};  // Example input
    int size = sizeof(nums) / sizeof(nums[0]);

    int thirdMax = findThirdDistinctMax(nums, size);

    if (thirdMax == nums[0]) {
```

```
        cout << "Maximum number: " << thirdMax << endl;
    } else {
        cout << "Third distinct maximum: " << thirdMax << endl;
    }

    return 0;
}
```

Explanation :

Insertion Sort:
We first sort the array in descending order using insertion sort. In insertion sort, each element is picked and placed in its correct position relative to the already sorted part of the array.
The sorting loop starts at the second element and compares it with previous elements, shifting elements to the right until it finds the correct position for the current element (key).

Finding Third Distinct Maximum:
After sorting, we traverse through the sorted array and count how many distinct elements we encounter.
Once we find the third distinct element, we break out of the loop.
If we don't find three distinct elements, we return the largest element (which is the first element in the sorted array)

Example:
Given the input {2, 2, 3, 1}:
The sorted array (in descending order) will be {3, 2, 2, 1}.
The distinct elements in this sorted array are 3, 2, and 1.
The third distinct maximum is 1, so the output will be:

10. A professor went to a party. Being an erudite person, he classified the party into two categories. He proposed that if all the persons in the party are wearing different colored robes, then that is a girl's only party. If we even get one duplicate color, it must be a boy's party. The colors of the robes are represented by positive integers. Input The first line of each test case contains an integer T, denoting the no of test cases. Then T test cases follow. The first line of each test case contains an integer N denoting the number of people in the party. In the next line are N space separated values of Ai denoting the color of the robes. Write the function for the above test case. OutputFor each test case, print "BOYS" without quotes if it's a boy's party, else print "GIRLS"

Here's a simple C++ solution to classify the party based on the colors of the robes as described. The approach is to use an array to track the occurrence of each color and determine if there are any duplicates.

```
#include <iostream>
using namespace std;

void classifyParty(int T) {
    for (int t = 0; t < T; t++) {
        int N;
        cin >> N;  // Read the number of people in the party
        int colors[101] = {0}; // Array to track robe colors (assuming color values are between 1 and 100)
        bool isBoysParty = false; // Flag to check for duplicate colors

        // Read the robe colors
        for (int i = 0; i < N; i++) {
            int color;
            cin >> color;
```

```
            // Increment the count for this color
            colors[color]++;

            // If the count of any color exceeds 1, mark as boys' party
            if (colors[color] > 1) {
                isBoysParty = true;
            }
        }

        // Output the result based on the flag
        if (isBoysParty) {
            cout << "BOYS" << endl;
        } else {
            cout << "GIRLS" << endl;
        }
    }
}

int main() {
    int T;
    cout << "Enter number of test cases: ";
    cin >> T; // Read the number of test cases

    classifyParty(T); // Call the function to classify the party
    return 0;
}
```

Explanation:

1. **Input Handling**:
   - The first input is the number of test cases `T`.
   - For each test case, read the number of people `N` and then the colors of the robes.

2. **Color Tracking**:
   - An array `colors` of size 101 is initialized to zero, assuming robe colors range from 1 to 100.
   - For each color read, increment the corresponding index in the `colors` array.

3. **Duplicate Checking**:
   - If any color count exceeds 1, it sets the `isBoysParty` flag to `true`, indicating that there is at least one duplicate color.

4. **Output**:
   - After processing all colors for the current test case, it checks the flag and prints "BOYS" or "GIRLS" accordingly.

Example Input and Output:

Input:

```
2
5
1 2 3 4 5
4
1 2 2 3
```

Output:
```
GIRLS
BOYS
```

In the first test case, all colors are distinct, so it's classified as a "GIRLS" party. In the second test case, the color `2` appears twice, classifying it as a "BOYS" party.

11. Write a function to merge two sorted doubly linked lists.
To merge two sorted doubly linked lists using logic similar to the merge step of merge sort, you can implement a function that combines the two lists in a way that maintains their sorted order. Below is the C++ implementation that follows this approach.

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node* prev;

    Node(int value) : data(value), next(nullptr), prev(nullptr) {}
};

class DoublyLinkedList {
public:
    Node* head;

    DoublyLinkedList() : head(nullptr) {}

    void append(int value) {
        Node* newNode = new Node(value);
        if (!head) {
            head = newNode;
            return;
        }
        Node* temp = head;
        while (temp->next) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }

    void printList() {
        Node* temp = head;
        while (temp) {
            std::cout << temp->data << " ";
            temp = temp->next;
        }
        std::cout << std::endl;
    }
};

// Merging two sorted doubly linked lists
DoublyLinkedList merge(DoublyLinkedList& list1, DoublyLinkedList& list2) {
```

```
    DoublyLinkedList mergedList;

    Node* p1 = list1.head;
    Node* p2 = list2.head;

    // Merging the two lists similar to merge sort logic
    while (p1 && p2) {
        if (p1->data < p2->data) {
            mergedList.append(p1->data);
            p1 = p1->next;
        } else {
            mergedList.append(p2->data);
            p2 = p2->next;
        }
    }

    // Append remaining elements of list1
    while (p1) {
        mergedList.append(p1->data);
        p1 = p1->next;
    }

    // Append remaining elements of list2
    while (p2) {
        mergedList.append(p2->data);
        p2 = p2->next;
    }

    return mergedList;
}

int main() {
    DoublyLinkedList list1;
    DoublyLinkedList list2;

    // Creating first sorted doubly linked list
    list1.append(1);
    list1.append(3);
    list1.append(5);

    // Creating second sorted doubly linked list
    list2.append(2);
    list2.append(4);
    list2.append(6);

    std::cout << "List 1: ";
    list1.printList();
    std::cout << "List 2: ";
    list2.printList();

    DoublyLinkedList mergedList = merge(list1, list2);
    std::cout << "Merged List: ";
    mergedList.printList();

    return 0;
}
```

```

### Explanation:

1. **Node Structure**: The `Node` structure defines each node in the doubly linked list, which includes data and pointers to the next and previous nodes.

2. **DoublyLinkedList Class**:
   - This class manages the linked list, allowing nodes to be appended and providing a method to print the list.
   - The `append` method adds a new node at the end of the list.

3. **Merge Function**:
   - The `merge` function combines two sorted doubly linked lists into one sorted doubly linked list.
   - Two pointers (`p1` for the first list and `p2` for the second list) traverse the lists. The smaller value from either list is appended to the merged list.
   - After processing both lists, any remaining nodes from either list are appended to the merged list.

4. **Main Function**:
   - It creates two sample sorted doubly linked lists, prints them, merges them, and finally prints the merged list.

### Example Output:
```
List 1: 1 3 5
List 2: 2 4 6
Merged List: 1 2 3 4 5 6
```

This implementation efficiently merges the two sorted doubly linked lists while maintaining the order of elements, similar to the merge operation in merge sort.

**12. Imagine a scenario of a lucky draw event and have a list of participants represented as a singly linked list. Write a function that retrieves the N-th participant from the end of the linked list, where N is a 1-based index. If N is greater than the number of participants or if the list is empty, the function should return None. ('N' will be taken from user)**
**Steps:**

1. **Two-pointer technique:**
   - **The first pointer is moved N steps ahead of the second pointer.**
   - **Then both pointers are moved together until the first pointer reaches the end.**
   - **The second pointer will now be at the N-th node from the end.**
2. **Edge Cases:**
   - **If the list is empty, return None (nullptr).**
   - **If N is greater than the size of the list, handle it gracefully.**

```
#include <iostream>
using namespace std;

// Define a class for the participant node
class Participant {
public:
   string name;
   Participant* next;
```
```

```
    Participant(string name) {
        this->name = name;
        this->next = nullptr;
    }
};

// Define a class for the linked list of participants
class ParticipantList {
public:
    Participant* head;

    ParticipantList() {
        head = nullptr;
    }

    // Function to add a participant to the end of the list
    void addParticipant(string name) {
        Participant* newParticipant = new Participant(name);
        if (head == nullptr) {
            head = newParticipant;
        } else {
            Participant* temp = head;
            while (temp->next != nullptr) {
                temp = temp->next;
            }
            temp->next = newParticipant;
        }
        cout << "Participant " << name << " added to the list." << endl;
    }

    // Function to retrieve the N-th participant from the end
    string nthParticipantFromEnd(int N) {
        if (head == nullptr) {
            return "List is empty.";
        }

        Participant* first = head;
        Participant* second = head;

        // Move the first pointer N steps ahead
        for (int i = 0; i < N; i++) {
            if (first == nullptr) {
                return "N is larger than the number of participants.";
            }
            first = first->next;
        }

        // Move both pointers together until the first reaches the end
        while (first != nullptr) {
            first = first->next;
            second = second->next;
        }

        // Now the second pointer points to the N-th node from the end
        return second->name;
```

```
    }
};

int main() {
    ParticipantList list;

    int choice, N;
    string name;

    do {
        cout << "\nMenu:\n";
        cout << "1. Add participant\n";
        cout << "2. Get N-th participant from the end\n";
        cout << "3. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter participant name: ";
                cin >> name;
                list.addParticipant(name);
                break;
            case 2:
                cout << "Enter the value of N: ";
                cin >> N;
                cout << "The " << N << "-th participant from the end is: " << list.nthParticipantFromEnd(N) <<
endl;
                break;
            case 3:
                cout << "Exiting...\n";
                break;
            default:
                cout << "Invalid choice. Try again.\n";
        }
    } while (choice != 3);

    return 0;
}
```

Class Participant:

- Represents each participant in the list with their name and a pointer next to the next participant.

Class ParticipantList:

- Contains the head of the linked list and methods to manipulate the list.
- addParticipant: Adds a participant to the end of the list.
- nthParticipantFromEnd: Retrieves the N-th participant from the end using the two-pointer technique.

Main function:

- Provides a menu to either add a participant or retrieve the N-th participant from the end of the list.

13. Suppose, you are developing a contact management system that stores contacts in a linked list. Each contact entry in the list includes details such as name, phone number, and email

address, and each contact node points to the next contact in the list. Find the total number of contacts in your list using a recursive method.

To count the total number of contacts in a linked list using recursion, you need a recursive function that traverses the list until it reaches the end. Each recursive call processes one contact and passes the rest of the list to the next recursive call, ultimately returning the count.

Steps:
1. Base case: If the list is empty (the current node is `nullptr`), return 0.
2. Recursive case: For each contact node, return `1 + count of the rest of the list`.

Here is a C++ implementation that demonstrates how to store contacts in a linked list and count the total number of contacts recursively:

C++ Implementation:

```cpp
#include <iostream>
using namespace std;

// Define a class for contact node
class Contact {
public:
    string name;
    string phone;
    string email;
    Contact* next;

    Contact(string name, string phone, string email) {
        this->name = name;
        this->phone = phone;
        this->email = email;
        this->next = nullptr;
    }
};

// Define a class for the contact list
class ContactList {
public:
    Contact* head;

    ContactList() {
        head = nullptr;
    }

    // Function to add a new contact at the end of the list
    void addContact(string name, string phone, string email) {
        Contact* newContact = new Contact(name, phone, email);
        if (head == nullptr) {
            head = newContact;
        } else {
            Contact* temp = head;
            while (temp->next != nullptr) {
                temp = temp->next;
            }
```

```
            temp->next = newContact;
        }
        cout << "Contact added: " << name << endl;
    }

    // Recursive function to count the total number of contacts
    int countContacts(Contact* node) {
        // Base case: if the current node is null, return 0
        if (node == nullptr) {
            return 0;
        }
        // Recursive case: 1 for this node + count of the rest of the list
        return 1 + countContacts(node->next);
    }

    // Function to start counting from the head
    int getTotalContacts() {
        return countContacts(head);
    }
};

int main() {
    ContactList contacts;

    contacts.addContact("Alice", "123-456-7890", "alice@example.com");
    contacts.addContact("Bob", "987-654-3210", "bob@example.com");
    contacts.addContact("Charlie", "555-555-5555", "charlie@example.com");

    cout << "Total contacts in the list: " << contacts.getTotalContacts() << endl;

    return 0;
}
```

Explanation:

1. **Class `Contact`**:
   - Stores each contact's `name`, `phone`, and `email`, along with a pointer `next` to the next contact.

2. **Class `ContactList`**:
   - Stores the head of the contact list.
   - `addContact`: Adds a new contact to the end of the linked list.
   - `countContacts`: A recursive function that counts the total number of contacts by traversing through the list.
   - `getTotalContacts`: This function initiates the recursive counting by passing the head of the list to the recursive function.

3. **Main function**:
   - Adds a few sample contacts and prints the total number of contacts using recursion.

Output:

```
Contact added: Alice
Contact added: Bob
Contact added: Charlie
```

Total contacts in the list: 3
```

### How the Recursive Function Works:
- Each call to `countContacts` checks if the current contact node is `nullptr` (the base case). If it is, the function returns 0.
- Otherwise, it returns `1 + countContacts(node->next)`, which adds 1 for the current node and recurses through the rest of the list.

**14. Write a function to search & remove duplicate entries from singly circular LL.**
To write a function that searches for and removes duplicate entries from a **singly circular linked list**, we need to:

1. Traverse the circular list.
2. For each node, check whether any duplicate nodes with the same value exist further down the list.
3. If duplicates are found, remove them.

**### C++ Implementation**

```cpp
#include<iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node(int val) {
        data = val;
        next = NULL;
    }
};

class CircularLinkedList {
    Node* head;

public:
    CircularLinkedList() {
        head = NULL;
    }

    // Function to insert a new node at the end of the list
    void insert(int val) {
        Node* newNode = new Node(val);

        if (head == NULL) {
            head = newNode;
            newNode->next = head;
        } else {
            Node* temp = head;
            while (temp->next != head) {
                temp = temp->next;
            }
            temp->next = newNode;
```
```

```
            newNode->next = head;
        }
    }

    // Function to display the circular linked list
    void display() {
        if (head == NULL) {
            cout << "List is empty!" << endl;
            return;
        }

        Node* temp = head;
        do {
            cout << temp->data << " -> ";
            temp = temp->next;
        } while (temp != head);
        cout << "(back to head)" << endl;
    }

    // Function to search and remove duplicate nodes
    void removeDuplicates() {
        if (head == NULL || head->next == head) {
            return;  // List is empty or has only one element
        }

        Node* current = head;

        do {
            Node* prev = current;
            Node* temp = current->next;

            // Check all subsequent nodes for duplicates
            while (temp != head) {
                if (temp->data == current->data) {
                    // Duplicate found, remove the node
                    prev->next = temp->next;
                    delete temp;
                    temp = prev->next;
                } else {
                    prev = temp;
                    temp = temp->next;
                }
            }

            current = current->next;
        } while (current != head);
    }
};

int main() {
    CircularLinkedList cll;

    // Adding some values to the circular linked list
    cll.insert(10);
    cll.insert(20);
    cll.insert(30);
```

```
    cll.insert(20);
    cll.insert(40);
    cll.insert(10);

    cout << "Original Circular Linked List:" << endl;
    cll.display();

    // Removing duplicates
    cll.removeDuplicates();

    cout << "\nCircular Linked List after removing duplicates:" << endl;
    cll.display();

    return 0;
}
```

Explanation:

1. **`insert(int val)`**: This function inserts a new node at the end of the circular linked list. It handles the case where the list is empty and connects the new node back to the head.

2. **`display()`**: This function traverses the circular list and prints the values of each node, indicating the circular nature of the list by showing `(back to head)` at the end.

3. **`removeDuplicates()`**:
   - **Outer loop (`current`)**: This loop starts from the head node and iterates through each node in the list.
   - **Inner loop (`temp`)**: For each node pointed to by `current`, this loop checks subsequent nodes to see if they contain the same value.
   - **Removing duplicates**: If a duplicate is found, the node is deleted, and the list is re-linked by setting the `next` pointer of the previous node (`prev`) to skip the duplicate node.

15. Write a function to detect loop or cycle in a singly linked list

To write a function that searches for and removes duplicate entries from a **singly circular linked list**, we need to:

1. Traverse the circular list.
2. For each node, check whether any duplicate nodes with the same value exist further down the list.
3. If duplicates are found, remove them.

C++ Implementation

```
#include<iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node(int val) {
        data = val;
        next = NULL;
    }
```

```
};

class CircularLinkedList {
    Node* head;

public:
    CircularLinkedList() {
        head = NULL;
    }

    // Function to insert a new node at the end of the list
    void insert(int val) {
        Node* newNode = new Node(val);

        if (head == NULL) {
            head = newNode;
            newNode->next = head;
        } else {
            Node* temp = head;
            while (temp->next != head) {
                temp = temp->next;
            }
            temp->next = newNode;
            newNode->next = head;
        }
    }

    // Function to display the circular linked list
    void display() {
        if (head == NULL) {
            cout << "List is empty!" << endl;
            return;
        }

        Node* temp = head;
        do {
            cout << temp->data << " -> ";
            temp = temp->next;
        } while (temp != head);
        cout << "(back to head)" << endl;
    }

    // Function to search and remove duplicate nodes
    void removeDuplicates() {
        if (head == NULL || head->next == head) {
            return;  // List is empty or has only one element
        }

        Node* current = head;

        do {
            Node* prev = current;
            Node* temp = current->next;

            // Check all subsequent nodes for duplicates
            while (temp != head) {
```

```
            if (temp->data == current->data) {
                // Duplicate found, remove the node
                prev->next = temp->next;
                delete temp;
                temp = prev->next;
            } else {
                prev = temp;
                temp = temp->next;
            }
        }

        current = current->next;
    } while (current != head);
  }
};

int main() {
  CircularLinkedList cll;

  // Adding some values to the circular linked list
  cll.insert(10);
  cll.insert(20);
  cll.insert(30);
  cll.insert(20);
  cll.insert(40);
  cll.insert(10);

  cout << "Original Circular Linked List:" << endl;
  cll.display();

  // Removing duplicates
  cll.removeDuplicates();

  cout << "\nCircular Linked List after removing duplicates:" << endl;
  cll.display();

  return 0;
}
```

Explanation:

1. **`insert(int val)`**: This function inserts a new node at the end of the circular linked list. It handles the case where the list is empty and connects the new node back to the head.

2. **`display()`**: This function traverses the circular list and prints the values of each node, indicating the circular nature of the list by showing `(back to head)` at the end.

3. **`removeDuplicates()`**:
   - **Outer loop (`current`)**: This loop starts from the head node and iterates through each node in the list.
   - **Inner loop (`temp`)**: For each node pointed to by `current`, this loop checks subsequent nodes to see if they contain the same value.
   - **Removing duplicates**: If a duplicate is found, the node is deleted, and the list is re-linked by setting the `next` pointer of the previous node (`prev`) to skip the duplicate node.

Example:

Input:
```
10 -> 20 -> 30 -> 20 -> 40 -> 10 -> (back to head)
```

Output:
```
10 -> 20 -> 30 -> 40 -> (back to head)
```

Explanation of Output:
- The duplicates `20` and `10` were found and removed. The list now contains only unique values.

16. You have a music playlist represented as a singly linked list. Each node in the linked list represents a song (title, artist, duration). Write a recursive function to find out whether a particular song is present in the playlist.
Here's the recursive function that checks whether a particular song is present in the playlist without using a `bool` return type. Instead, we'll print whether the song is found directly from the recursive function.

Updated Code:

```cpp
#include <iostream>
using namespace std;

class Song {
   string title;
   string artist;
   string duration;

public:
   Song *next;
   Song() { next = NULL; }

   void read() {
      cout << "Enter song title: ";
      cin >> title;
      cout << "Enter artist name: ";
      cin >> artist;
      cout << "Enter duration: ";
      cin >> duration;
   }

   string getTitle() { return title; }
};

class Playlist {
   Song *head;

public:
   Playlist() { head = NULL; }

   void addSong() {
      Song *nn = new Song;
```

```
        nn->read();
        if (head == NULL) {
            head = nn;
        } else {
            Song *cn = head;
            while (cn->next != NULL) {
                cn = cn->next;
            }
            cn->next = nn;
        }
    }

    // Recursive function to search for a song by title
    void findSongRecursive(Song *node, const string &title) {
        if (node == NULL) {
            cout << "Song not found in the playlist." << endl;
            return; // Base case: reached the end of the list
        }
        if (node->getTitle() == title) {
            cout << "Song found in the playlist!" << endl;
            return; // Song found
        }
        findSongRecursive(node->next, title); // Recursive case
    }

    void searchSong() {
        if (head == NULL) {
            cout << "Playlist is empty." << endl;
            return;
        }

        string title;
        cout << "Enter the title of the song to search: ";
        cin >> ws;
        getline(cin, title); // Allow for multi-word titles

        findSongRecursive(head, title); // Call the recursive function
    }
};

int main() {
    Playlist myPlaylist;
    int choice;

    do {
        cout << "\nPlaylist Menu:\n";
        cout << "1. Add song\n";
        cout << "2. Search for a song\n";
        cout << "3. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            myPlaylist.addSong();
            break;
```

```
        case 2:
            myPlaylist.searchSong();
            break;
        case 3:
            cout << "Exiting..." << endl;
            break;
        default:
            cout << "Invalid choice, please try again." << endl;
        }
    } while (choice != 3);

    return 0;
}
```

Explanation:
- **`findSongRecursive()`**: This is the recursive function that searches for the song in the playlist. Instead of returning `true` or `false`, it directly prints if the song is found or not within the function. If the title matches, it prints "Song found in the playlist!", and if it reaches the end of the list without finding the song, it prints "Song not found in the playlist."

- **`searchSong()`**: This function initiates the search process by prompting the user for the title of the song and calling `findSongRecursive()` to perform the actual search.

Menu:
The main program also provides a simple menu:
1. Add a song to the playlist.
2. Search for a specific song by title.
3. Exit the program.

This version doesn't use a `bool` for the search logic but instead handles everything with output statements from the recursive function itself.

17. You are given a singly linked list where each node contains the following information: data (an integer representing the value of the node) next (a reference to the next node in the list) Your task is to write a function that retrieves the value of the N-th node in the linked list. If N is greater than the length of the list or if the list is empty, the function should handle these cases appropriately.

To retrieve the value of the N-th node in a singly linked list, we'll write a function that traverses the list and returns the value of the N-th node (assuming 1-based indexing). If N is greater than the length of the list or if the list is empty, we'll handle the error cases accordingly.

Steps:

1. Traverse the list node by node, and keep track of the current position.
2. If we reach the N-th node, return its value.
3. If the list is shorter than N or is empty, indicate that the position is invalid.

Code Implementation:

cpp
Copy code
```
#include <iostream>
using namespace std;
```

```
class Node {
public:
    int data;
    Node* next;

    Node(int val) {
        data = val;
        next = NULL;
    }
};

class LinkedList {
    Node* head;

public:
    LinkedList() {
        head = NULL;
    }

    // Function to add a node at the end of the list
    void append(int value) {
        Node* newNode = new Node(value);
        if (head == NULL) {
            head = newNode;
        } else {
            Node* temp = head;
            while (temp->next != NULL) {
                temp = temp->next;
            }
            temp->next = newNode;
        }
    }

    // Function to retrieve the N-th node's value (1-based index)
    void getNthNode(int N) {
        if (head == NULL) {
            cout << "The list is empty." << endl;
            return;
        }

        Node* temp = head;
        int count = 1; // Start counting from the first node

        // Traverse the list to find the N-th node
        while (temp != NULL) {
            if (count == N) {
                cout << "The value of the " << N << "-th node is: " << temp->data << endl;
                return;
            }
            temp = temp->next;
            count++;
        }

        // If we reach here, N is greater than the length of the list
        cout << "Invalid position. The list has fewer than " << N << " nodes." << endl;
    }
```

```
    // Function to display the entire list
    void display() {
        if (head == NULL) {
            cout << "The list is empty." << endl;
            return;
        }

        Node* temp = head;
        while (temp != NULL) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {
    LinkedList myList;
    int choice, value, N;

    do {
        cout << "\nMenu:\n";
        cout << "1. Append a node\n";
        cout << "2. Retrieve the N-th node's value\n";
        cout << "3. Display the list\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            cout << "Enter a value to append: ";
            cin >> value;
            myList.append(value);
            break;
        case 2:
            cout << "Enter the position of the node to retrieve (N): ";
            cin >> N;
            myList.getNthNode(N);
            break;
        case 3:
            myList.display();
            break;
        case 4:
            cout << "Exiting..." << endl;
            break;
        default:
            cout << "Invalid choice, please try again." << endl;
        }
    } while (choice != 4);

    return 0;
}
```

Explanation:

- append(int value): Adds a new node to the end of the list. If the list is empty, the new node becomes the head.
- getNthNode(int N): This function retrieves the value of the N-th node in the list. It starts at the head and traverses the list, counting nodes. If N exceeds the length of the list, it prints an appropriate message. If the list is empty, it handles that case as well.
- display(): Displays the entire list for verification purposes.

Menu:

1. Append a node: Adds a new node with an integer value to the list.
2. Retrieve the N-th node's value: Asks the user to input a position N and returns the value of the node at that position.
3. Display the list: Prints the entire linked list.
4. Exit: Ends the program.

18. You are organizing a lucky draw event and have a list of participants represented as a singly linked list. Each node in the linked list contains: name (a string representing the participant's name) next (a reference to the next node in the list) Your task is to write a function that retrieves the N-th participant from the linked list, where N is a 1-based Index. If N is greater than the number of participants or if the list is empty, the function should return None.
To retrieve the N-th participant in a singly linked list of participants, we'll write a function that traverses the list and returns the name of the N-th participant (using 1-based indexing). If `N` is greater than the number of participants or if the list is empty, the function will return `None` to indicate that the position is invalid.

Approach:
1. Traverse the list, keeping track of the current position.
2. If the position matches `N`, return the participant's name.
3. If `N` is greater than the length of the list or the list is empty, return `None`.

Code Implementation:

```cpp
#include <iostream>
#include <string>
using namespace std;

class Participant {
public:
   string name;
   Participant* next;

   Participant(string participantName) {
      name = participantName;
      next = NULL;
   }
};

class LuckyDraw {
   Participant* head;

public:
```

```
    LuckyDraw() {
        head = NULL;
    }

    // Function to add a participant at the end of the list
    void addParticipant(string participantName) {
        Participant* newParticipant = new Participant(participantName);
        if (head == NULL) {
            head = newParticipant;
        } else {
            Participant* temp = head;
            while (temp->next != NULL) {
                temp = temp->next;
            }
            temp->next = newParticipant;
        }
    }

    // Function to retrieve the N-th participant's name (1-based index)
    string getNthParticipant(int N) {
        if (head == NULL) {
            return "The list is empty.";
        }

        Participant* temp = head;
        int count = 1; // Start counting from the first participant

        // Traverse the list to find the N-th participant
        while (temp != NULL) {
            if (count == N) {
                return temp->name;
            }
            temp = temp->next;
            count++;
        }

        // If we reach here, N is greater than the number of participants
        return "Invalid position. There are fewer than " + to_string(N) + " participants.";
    }

    // Function to display all participants
    void displayParticipants() {
        if (head == NULL) {
            cout << "The participant list is empty." << endl;
            return;
        }

        Participant* temp = head;
        while (temp != NULL) {
            cout << temp->name << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};
```

```
int main() {
    LuckyDraw event;
    int choice, N;
    string participantName;

    do {
        cout << "\nLucky Draw Menu:\n";
        cout << "1. Add participant\n";
        cout << "2. Retrieve N-th participant\n";
        cout << "3. Display all participants\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            cout << "Enter participant name: ";
            cin >> ws;  // To consume any leading whitespace
            getline(cin, participantName);
            event.addParticipant(participantName);
            break;
        case 2:
            cout << "Enter the position (N) of the participant to retrieve: ";
            cin >> N;
            cout << event.getNthParticipant(N) << endl;
            break;
        case 3:
            event.displayParticipants();
            break;
        case 4:
            cout << "Exiting..." << endl;
            break;
        default:
            cout << "Invalid choice, please try again." << endl;
        }
    } while (choice != 4);

    return 0;
}
```

Explanation:
- **`addParticipant(string participantName)`**: Adds a new participant (node) to the end of the list. If the list is empty, the new participant becomes the head.

- **`getNthParticipant(int N)`**: This function retrieves the name of the N-th participant. If `N` is greater than the length of the list or if the list is empty, it returns an appropriate message.

- **`displayParticipants()`**: Displays all participants in the list for verification purposes.

Menu:
1. **Add participant**: Adds a new participant by asking the user for their name.
2. **Retrieve N-th participant**: Asks the user to input the position `N` and returns the name of the participant at that position.
3. **Display all participants**: Prints the entire list of participants.
4. **Exit**: Exits the program.

This function handles cases where the list is empty or where `N` exceeds the number of participants, providing appropriate feedback to the user.

19. Given a singly linked list, write a function to find the middle element of the list.
Find Middle of the Linked List
Last Updated : 04 Sep, 2024
Given a singly linked list, the task is to find the middle of the linked list. If the number of nodes are even, then there would be two middle nodes, so return the second middle node.

Example:

Input: linked list: 1->2->3->4->5
Output: 3
Explanation: There are 5 nodes in the linked list and there is one middle node whose value is 3.

Input: linked list = 10 -> 20 -> 30 -> 40 -> 50 -> 60
Output: 40
Explanation: There are 6 nodes in the linked list, so we have two middle nodes: 30 and 40, but we will return the second middle node which is 40.

```
// C++ program to illustrate how to find the middle element
// by counting the number of nodes
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node(int x) {
        this->data = x;
        this->next = nullptr;
    }
};

// Helper function to find length of linked list
int getLength(Node* head) {

    // Variable to store length
    int length = 0;

    // Traverse the entire linked list and increment length by
    // 1 for each node
    while (head) {
        length++;
        head = head->next;
    }

    // Return the number of nodes in the linked list
    return length;
}
```

```
// Function to find the middle element of the linked list
int getMiddle(Node* head) {

    // Finding length of the linked list
    int length = getLength(head);

    // traverse till we reached half of length
    int mid_index = length / 2;
    while (mid_index--) {
        head = head->next;
    }

    // Head now will be pointing to the middle element
    return head->data;
}

int main() {

    // Create a hard-coded linked list:
    // 10 -> 20 -> 30 -> 40 -> 50 -> 60
    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->next = new Node(30);
    head->next->next->next = new Node(40);
    head->next->next->next->next = new Node(50);
    head->next->next->next->next->next = new Node(60);

    cout << getMiddle(head);

    return 0;
}
```

Output
40
Time Complexity: O(2 * n) = O(n) where n is the number of nodes in the linked list.
Auxiliary Space: O(1)

20. Write a function to swap nodes in a doubly linked list without swapping data.
Given a doubly linked list having all unique elements and two keys X and Y, the task is to swap nodes for two given keys by changing links only.

Note: It may be considered that X and Y are always present in the list.

Examples:

Input: list = 1 <-> 8 <-> 7 <-> 9 <-> 4, X = 1, Y = 4
Output: 4 <-> 8 <-> 7 <-> 9 <-> 1

Input: list = 0 <-> 1, X = 0, Y = 1
Output: 1 <-> 0

Approach: The problem can be solved by traversing pointers to nodes having values X and Y and swap them.

Follow the steps mentioned below:

Search X and Y in the given doubly linked list.
After searching, swap the nodes by making the previous adjacent pointer of X to be previous adjacent pointer of Y and next adjacent pointer of X to be next adjacent pointer of Y and vice versa.
Below is the implementation of the above approach:

// C++ code to implement the above approach

```
#include <bits/stdc++.h>
using namespace std;

// Link list Node Class
class Node {
public:
   int data;
   Node* prev;
   Node* next;

   // Constructor function
   Node(int data)
   {
      this->data = data;
      this->prev = NULL;
      this->next = NULL;
   }
};

// Function to print linked list
void print(Node* head)
{
   Node* temp = head;

   // Iterate until node is NOT NULL
   while (temp != NULL) {
      cout << temp->data << " ";
      temp = temp->next;
   }
   cout << endl;
}

// Function to push a node in DLL
void push(Node*& head, Node*& tail,
         int item)
{

   // DLL is empty
   if (tail == NULL) {

      Node* temp = new Node(item);
      tail = temp;
      head = temp;
   }
```

```
    // DLL is not empty
    else {
        Node* temp = new Node(item);
        tail->next = temp;
        temp->prev = tail;
        tail = temp;
    }
}

// Function to find the nodes
// which have to be swapped
pair<Node*, Node*> find(Node*& head,
                        int x, int y)
{
    Node* N1 = NULL;
    Node* N2 = NULL;
    Node* temp = head;

    // Traversing the list
    while (temp != NULL) {
        if (temp->data == x)
            N1 = temp;
        else if (temp->data == y)
            N2 = temp;
        temp = temp->next;
    }
    return make_pair(N1, N2);
}

// Function to swap the nodes
// consisting of x and y
void swap(Node*& head, Node*& tail,
          int x, int y)
{

    // Edge Cases
    if (head == NULL || head->next == NULL
        || x == y)
        return;

    // Finding the Nodes
    pair<Node*, Node*> p = find(head, x, y);

    Node* Node1 = p.first;
    Node* Node2 = p.second;

    if (Node1 == head)
        head = Node2;
    else if (Node2 == head)
        head = Node1;
    if (Node1 == tail)
        tail = Node2;
    else if (Node2 == tail)
        tail = Node1;
```

```
    // Swapping Node1 and Node2
    Node* temp;
    temp = Node1->next;
    Node1->next = Node2->next;
    Node2->next = temp;

    if (Node1->next != NULL)
        Node1->next->prev = Node1;
    if (Node2->next != NULL)
        Node2->next->prev = Node2;

    temp = Node1->prev;
    Node1->prev = Node2->prev;
    Node2->prev = temp;

    if (Node1->prev != NULL)
        Node1->prev->next = Node1;
    if (Node2->prev != NULL)
        Node2->prev->next = Node2;
}

// Driver Code
int main()
{

    Node* head = NULL;
    Node* tail = NULL;

    push(head, tail, 1);
    push(head, tail, 8);
    push(head, tail, 7);
    push(head, tail, 9);
    push(head, tail, 4);

    int X = 1, Y = 4;

    cout << "Before Swapping: ";
    print(head);

    swap(head, tail, X, Y);
    cout << "After Swapping: ";
    print(head);

    return 0;
}
```

Output
Before Swapping: 1 8 7 9 4
After Swapping: 4 8 7 9 1

Time Complexity: O(N)
Auxiliary Space: O(1)

21. Print the middle of a given linked list without counting the total number of nodes

CODE:

```
/*print the middle element of the given singly linkedlist without counting the total number of nodes */
/*To find the middle element of a singly linked list without counting the total number of nodes, we can use
the "two-pointer" approach (also known as the "tortoise and hare" algorithm). Here's how it works:

-Maintain two pointers: one (slow_ptr) and one (fast_ptr).
-Move slow_ptr by one step and fast_ptr by two steps.
-When fast_ptr reaches the end of the list, slow_ptr will be at the middle of the list.*/

#include <iostream>
using namespace std;

class Node
{
	public:
	int no;
	Node *next;
	Node()
	{
		no = 0;
		next = NULL;
	}
};

class linkedlist
{
	Node *start;

	public:
	linkedlist()
	{
		start = NULL;
	}

	void create()
	{
		Node *nnode;
		Node *temp;
		char ch;
		do{
			nnode = new Node;
			cout<<"Enter the number to be stored: ";
			cin>>nnode -> no;

			if(start == NULL)
			{
				start = nnode;
			}
			else
			{
				temp = start;
				while (temp -> next != NULL)
					temp = temp -> next;
				temp -> next = nnode;
```

```
                        }
                        cout<<"Do you have more data (y/n)?? :";
                                cin>>ch;
                }
                while (ch == 'y' || ch == 'Y');
        }

    void middle()
    {
        Node *slow_ptr = start;
        Node *fast_ptr = start;

         if (start != nullptr)
        {
            while (fast_ptr != nullptr && fast_ptr->next != nullptr)
            {
                slow_ptr = slow_ptr->next;
                fast_ptr = fast_ptr->next->next;
            }
            cout << "The middle element is: " << slow_ptr->no << endl;
        }
    }

        void display()
        {
                Node *temp;
                cout<<"\nThe store numbers are: ";
                for (temp = start;temp != NULL;temp = temp -> next)
                {
                        cout<<temp -> no<<'\t';
                }
        }
};



int main()
{
        linkedlist l;
        l.create();
    l.middle();
        l.display();;
        return 0;
}
```

22. Write a function that counts the number of times a given int occurs in a Linked List

CODE:

/*write a function that counts the number of times a given int occurs in a linked list in c++*/
/*he countOccurrences() function traverses the list and counts how many times the specified integer value appears:
It iterates through the list starting from the head.
For each node, it checks if the node's data matches the given value.
If it does, it increments the count*/

```
/*Creates a linked list and appends values: 1 -> 2 -> 3 -> 2 -> 2 -> 4.
Calls the countOccurrences() function to count how many times the number 2 appears in the list (which is
3 times).
Prints the result.*/

#include <iostream>
using namespace std;

class Node
{
	public:
	int no;
	Node *next;
	Node()
	{
		no = 0;
		next = NULL;
	}
};

class linkedlist
{
	Node *start;

	public:
	linkedlist()
	{
		start = NULL;
	}

	void create()
	{
		Node *nnode;
		Node *temp;
		char ch;
		do{
			nnode = new Node;
			cout<<"Enter the number to be stored: ";
			cin>>nnode -> no;

			if(start == NULL)
			{
				start = nnode;
			}
			else
			{
				temp = start;
				while (temp -> next != NULL)
					temp = temp -> next;
				temp -> next = nnode;
			}
			cout<<"Do you have more data (y/n)?? : ";
				cin>>ch;
		}
		while (ch == 'y');
	}
```

```
    // Function to count occurrences of a given integer
   int countOccurrences(int value)
   {
      Node* temp = start;
      int count = 0;

      while (temp != nullptr)
      {
         if (temp->no == value)
         {
            count++;
         }
         temp = temp->next;
      }

      return count;
   }

         void display()
         {
                  Node *temp;
                  cout<<"\nThe store numbers are: ";
                  for (temp = start;temp != NULL;temp = temp -> next)
                  {
                           cout<<temp -> no<<'\t';
                  }
         }
};

int main()
{
   int n;
   linkedlist l;

   l.create();
   cout<<"\nEnter which int no you have to count: ";
   cin>>n;
   int x = l.countOccurrences(n);
   cout<<"The count of "<<n<<" is: "<<x<<endl;
         l.display();
         return 0;
}
```

23. Detect loop in a linked list

CODE:

```
/*Detecting a loop in a linked list is a common problem in data structures.
The most efficient solution to detect a loop uses "Floyd's Cycle-Finding Algorithm",
also known as the "Tortoise and Hare" approach*/

#include <iostream>
using namespace std;
```

```
class Node
{
	public:
	int no;
	Node *next;
	Node()
	{
		no = 0;
		next = NULL;
	}
};

class linkedlist
{
	Node *start;

	public:
	linkedlist()
	{
		start = NULL;
	}

	void create()
	{
		Node *nnode;
		Node *temp;
		char ch;
		do{
			nnode = new Node;
			cout<<"Enter the number to be stored: ";
			cin>>nnode -> no;

			if(start == NULL)
			{
				start = nnode;
			}
			else
			{
				temp = start;
				while (temp -> next != NULL)
					temp = temp -> next;
				temp -> next = nnode;
			}
			cout<<"Do you have more data (y/n)?? :";
				cin>>ch;
		}
		while (ch == 'y' || ch == 'Y');
	}

   bool detect_loop()
   {
      Node *slow_ptr = start;
      Node *fast_ptr = start;

      while (fast_ptr != nullptr && fast_ptr->next != nullptr)
      {
```

```
        slow_ptr = slow_ptr->next;
        fast_ptr = fast_ptr->next->next;

        // If the slow and fast pointers meet, a loop is detected
        if (slow_ptr == fast_ptr)
        {
            return true;
        }
    }

    // No loop detected
    return false;
  }

        void display()
        {
                Node *temp;
                cout<<"\nThe store numbers are: ";
                for (temp = start;temp != NULL;temp = temp -> next)
                {
                        cout<<temp -> no<<'\t';
                }
        }
};



int main()
{
        linkedlist l;
        l.create();
  if(l.detect_loop())
  {
    cout<<"Loop detected in the linked list" << endl;
  }
  else
  {
    cout << "No loop detected in the linked list" << endl;
  }
        l.display();
        return 0;
}
```

24. Find length of loop in linked list
To find the length of a loop in a linked list, we can use Floyd's Cycle-Finding Algorithm (also known as the Tortoise and Hare algorithm). Here's the approach:

Steps:
1. **Detect the loop**: Use two pointers (slow and fast). Slow moves one step at a time, while fast moves two steps at a time. If there is a loop, the slow and fast pointers will eventually meet.
2. **Find the length of the loop**: Once a loop is detected, keep one pointer at the meeting point and move the other pointer around the loop until it meets the first pointer again. Count the number of steps it takes to complete this cycle, which will give you the length of the loop.

Code Implementation:

```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node(int val) {
        data = val;
        next = NULL;
    }
};

// Function to detect the loop and return its length
int findLoopLength(Node* head) {
    Node* slow = head;
    Node* fast = head;

    // Step 1: Detect loop using Floyd's Cycle-Finding Algorithm
    while (slow != NULL && fast != NULL && fast->next != NULL) {
        slow = slow->next;         // Move slow pointer by 1 step
        fast = fast->next->next;   // Move fast pointer by 2 steps

        // If slow and fast meet, there is a loop
        if (slow == fast) {
            // Step 2: Find loop length
            int loopLength = 1;
            Node* temp = slow;

            // Move temp pointer around the loop to count the number of nodes in the loop
            while (temp->next != slow) {
                loopLength++;
                temp = temp->next;
            }

            return loopLength;  // Return the length of the loop
        }
    }

    // If no loop found, return 0
    return 0;
}

// Helper function to create a loop for testing purposes
void createLoop(Node* head, int position) {
    if (position == 0) return;

    Node* temp = head;
    Node* loopStart = NULL;
    int count = 1;

    // Traverse the list to find the position where the loop should start
    while (temp->next != NULL) {
```

```
        if (count == position) {
            loopStart = temp;
        }
        temp = temp->next;
        count++;
    }

    // Connect the last node to the node at 'position' to create a loop
    temp->next = loopStart;
}

int main() {
    // Creating a linked list with a loop for testing
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
    head->next->next->next->next = new Node(5);

    // Create a loop for testing (loop starting at node with value 3)
    createLoop(head, 3);

    // Find and print the length of the loop
    int loopLength = findLoopLength(head);
    if (loopLength > 0) {
        cout << "Loop detected! Length of the loop: " << loopLength << endl;
    } else {
        cout << "No loop detected in the linked list." << endl;
    }

    return 0;
}
```

Explanation:
- **findLoopLength()**: This function uses the Floyd’s Cycle-Finding Algorithm to detect a loop in the linked list. If a loop is detected, it calculates the loop's length by counting the number of nodes in the cycle.

- **createLoop()**: This helper function is used to create a loop in the linked list for testing purposes. It connects the last node to a specific node (given by `position`) to form a loop.

Output:

```
Loop detected! Length of the loop: 3
```

In this example, the loop starts at the node with value 3 and the length of the loop is 3 nodes. If no loop is detected, the program will output `No loop detected in the linked list.`.

25. Function to check if a singly linked list is palindrome

CODE:
```
/*Fuction to check if the singly linkedlist is palindrome*/
#include <iostream>
```

```
using namespace std;

class Node
{
public:
    int no;
    Node *next;
    Node()
    {
        no = 0;
        next = NULL;
    }
};

class linkedlist
{
    Node *start;

public:
    linkedlist()
    {
        start = NULL;
    }

    void create()
    {
        Node *nnode;
        Node *temp;
        char ch;
        do
        {
            nnode = new Node;
            cout << "Enter the number to be stored: ";
            cin >> nnode->no;

            if (start == NULL)
            {
                start = nnode;
            }
            else
            {
                temp = start;
                while (temp->next != NULL)
                    temp = temp->next;
                temp->next = nnode;
            }
            cout << "Do you have more data (y/n)?? :";
            cin >> ch;
        } while (ch == 'y' || ch == 'Y');
    }

    Node* reverse(Node* head)
    {
        Node *prev = NULL, *current = head, *next = NULL;
        while (current != NULL)
        {
```

```
        next = current->next; // Store next
        current->next = prev; // Reverse current node's pointer
        prev = current;       // Move pointers one position ahead.
        current = next;
    }
    return prev;
}

bool isPalindrome()
{
    if (start == NULL || start->next == NULL)
        return true;

    // Step 1: Find the middle of the linked list
    Node *slow = start, *fast = start;
    while (fast != NULL && fast->next != NULL)
    {
        slow = slow->next;
        fast = fast->next->next;
    }

    // Step 2: Reverse the second half
    Node *secondHalfStart = reverse(slow);

    // Step 3: Compare the first half with the reversed second half
    Node *firstHalfStart = start;
    Node *secondHalfIter = secondHalfStart;
    while (secondHalfIter != NULL)
    {
        if (firstHalfStart->no != secondHalfIter->no)
        {
            return false;
        }
        firstHalfStart = firstHalfStart->next;
        secondHalfIter = secondHalfIter->next;
    }

    // Optional: Restore the list by reversing the second half again (to maintain the original list structure)
    reverse(secondHalfStart);

    return true;
}

void display()
{
    Node *temp;
    cout << "\nThe stored numbers are: ";
    for (temp = start; temp != NULL; temp = temp->next)
    {
        cout << temp->no << '\t';
    }
    cout << endl;
}
};

int main()
```

```
{
    linkedlist l;
    l.create();

    l.display(); // Display the list

    if (l.isPalindrome())
    {
        cout << "\nThe list is a palindrome." << endl;
    }
    else
    {
        cout << "\nThe list is not a palindrome." << endl;
    }


    return 0;
}
```

26. Remove duplicates from a sorted linked list

CODE:

```
/*Remove duplicates from a sorted linked list*/

#include <iostream>
#include <unordered_set>
using namespace std;

class Node {
public:
    int data;      // Data stored in the node
    Node* next;    // Pointer to the next node
    Node(int value) {
        data = value;
        next = nullptr;
    }
};

class LinkedList {
    Node* head;   // Pointer to the first node

public:
    LinkedList() {
        head = nullptr;
    }

    // Function to create the linked list
    void create() {
        Node* newNode;
        Node* temp;
        char choice;

        do {
            int value;
            cout << "Enter the number for sorted linkedlist: ";
            cin >> value;
```

```
        newNode = new Node(value);

        if (head == nullptr) {
            head = newNode;
        } else {
            temp = head;
            while (temp->next != nullptr)
                temp = temp->next;
            temp->next = newNode;
        }

        cout << "Do you have more data (y/n)? : ";
        cin >> choice;

    } while (choice == 'y' || choice == 'Y');
}

// Function to remove duplicates from a sorted linked list
void removeDuplicates() {
    if (head == nullptr || head->next == nullptr)
        return;

    Node* current = head;
    while (current->next != nullptr) {
        if (current->data == current->next->data) {
            Node* duplicateNode = current->next;
            current->next = current->next->next;
            delete duplicateNode;
        } else {
            current = current->next;
        }
    }
}


// Function to display the linked list
void display() {
    Node* temp = head;
    cout << "\nThe stored numbers are: ";
    while (temp != nullptr) {
        cout << temp->data << '\t';
        temp = temp->next;
    }
    cout << endl;
}
};

int main() {
    LinkedList list;

    // Create the linked list
    list.create();

    // Display the original list
    list.display();
```

```
    // Remove duplicates from the list
    list.removeDuplicates();

    // Display the list after removing duplicates
    list.display();


    return 0;
}
```

27. Given a doubly linked list, write a function to detect if the list is a palindrome.

CODE:

```
#include <iostream>
using namespace std;

// Class for Node
class Node {
public:
    char data;
    Node* next;
    Node* prev;

    // Constructor to initialize a new node
    Node(char new_data) {
        data = new_data;
        next = nullptr;
        prev = nullptr;
    }
};

// Function to insert a new node at the front of the list
void push(Node** head_ref, char new_data) {
    Node* new_node = new Node(new_data);
    new_node->next = (*head_ref);
    new_node->prev = nullptr;

    if ((*head_ref) != nullptr)
        (*head_ref)->prev = new_node;

    (*head_ref) = new_node;
}

// Function to check if the list is a palindrome or not
bool isPalindrome(Node* left) {
    if (left == nullptr)
        return true;

    // Find the rightmost node
    Node* right = left;
    while (right->next != nullptr)
        right = right->next;

    while (left != right) {
        if (left->data != right->data)
```

```
            return false;

        if (left->next == right)
            break;

        left = left->next;
        right = right->prev;
    }

    return true;
}

// Driver program
int main() {
    Node* head = nullptr;

    // Adding characters to the doubly linked list
    push(&head, 'l');
    push(&head, 'e');
    push(&head, 'v');
    push(&head, 'e');
    push(&head, 'l');

    // Checking if the doubly linked list is a palindrome
    if (isPalindrome(head))
        cout << "It is a Palindrome" << endl;
    else
        cout << "Not a Palindrome" << endl;

    return 0;
}
```

Output
It is Palindrome

Time complexity: O(n)
Auxiliary Space : O(1)

28. Write a function to organize a music playlist using a linked list structure, where each song is categorized based on its play count. Create two separate lists: one for songs with an even number of plays and another for songs with an odd number of plays. Ensure that the system preserves the original order of songs within each list.

To organize a music playlist using a linked list structure where each song is categorized based on its play count, we can create two separate linked lists: one for songs with an even number of plays and another for songs with an odd number of plays. The implementation will ensure that the original order of songs is preserved in each list.

CODE:

```
#include <iostream>
#include <string>
using namespace std;
```

```
// Class for Song
class Song {
public:
    string title;
    string artist;
    int playCount;
    Song* next;

    // Constructor
    Song(string t, string a, int p) : title(t), artist(a), playCount(p), next(nullptr) {}
};

// Class for Playlist
class Playlist {
private:
    Song* head;

public:
    Playlist() : head(nullptr) {}

    // Function to add a song to the playlist
    void addSong(string title, string artist, int playCount) {
        Song* newSong = new Song(title, artist, playCount);
        if (head == nullptr) {
            head = newSong;
        } else {
            Song* temp = head;
            while (temp->next != nullptr) {
                temp = temp->next;
            }
            temp->next = newSong;
        }
    }

    // Function to organize songs based on play count
    void organizePlaylist(Playlist& evenPlaylist, Playlist& oddPlaylist) {
        Song* current = head;
        while (current != nullptr) {
            if (current->playCount % 2 == 0) {
                evenPlaylist.addSong(current->title, current->artist, current->playCount);
            } else {
                oddPlaylist.addSong(current->title, current->artist, current->playCount);
            }
            current = current->next;
        }
    }

    // Function to display the playlist
    void displayPlaylist() {
        if (head == nullptr) {
            cout << "Playlist is empty." << endl;
            return;
        }
        Song* current = head;
        while (current != nullptr) {
```

```
            cout << "Title: " << current->title << ", Artist: " << current->artist << ", Plays: " <<
current->playCount << endl;
            current = current->next;
        }
    }
};

int main() {
    Playlist myPlaylist;

    // Adding songs to the playlist
    myPlaylist.addSong("Song 1", "Artist A", 3);
    myPlaylist.addSong("Song 2", "Artist B", 4);
    myPlaylist.addSong("Song 3", "Artist C", 7);
    myPlaylist.addSong("Song 4", "Artist D", 2);
    myPlaylist.addSong("Song 5", "Artist E", 5);

    cout << "Original Playlist:" << endl;
    myPlaylist.displayPlaylist();

    Playlist evenPlaylist, oddPlaylist;
    myPlaylist.organizePlaylist(evenPlaylist, oddPlaylist);

    cout << "\nEven Play Count Playlist:" << endl;
    evenPlaylist.displayPlaylist();

    cout << "\nOdd Play Count Playlist:" << endl;
    oddPlaylist.displayPlaylist();

    return 0;
}
```

Explanation of the Code

1. **Song Class**:
   - Represents each song with `title`, `artist`, `playCount`, and a pointer to the next song (`next`).

2. **Playlist Class**:
   - Contains a pointer to the head of the list (`head`).
   - The `addSong()` method allows adding a new song to the playlist.
   - The `organizePlaylist()` method iterates through the songs in the original playlist and categorizes them into even and odd play count playlists by calling the `addSong()` method of the respective playlists.

3. **Display Function**:
   - The `displayPlaylist()` method prints all songs in the current playlist.

4. **Main Function**:
   - Creates an instance of the original playlist, adds several songs, and then organizes the songs into even and odd playlists.
   - Displays the original playlist followed by the organized even and odd playlists.

Output
When you run the code, it will output the original playlist followed by the even and odd playlists:

```
```

Original Playlist:
Title: Song 1, Artist: Artist A, Plays: 3
Title: Song 2, Artist: Artist B, Plays: 4
Title: Song 3, Artist: Artist C, Plays: 7
Title: Song 4, Artist: Artist D, Plays: 2
Title: Song 5, Artist: Artist E, Plays: 5

Even Play Count Playlist:
Title: Song 2, Artist: Artist B, Plays: 4
Title: Song 4, Artist: Artist D, Plays: 2

Odd Play Count Playlist:
Title: Song 1, Artist: Artist A, Plays: 3
Title: Song 3, Artist: Artist C, Plays: 7
Title: Song 5, Artist: Artist E, Plays: 5
```

This code organizes the songs based on their play counts while preserving their original order in the respective lists.

**29. Remove duplicates from an unsorted linked list**
**Swapping nodes in a linked list without swapping data requires manipulating the pointers of the nodes directly. Here's how you can achieve this in C++:**

**CODE:**
```
/*Remove duplicates from a unsorted linked list*/

#include <iostream>
using namespace std;

// Node structure definition
class Node {
public:
   int data;   // Data stored in the node
   Node* next;   // Pointer to the next node

   // Constructor to initialize a node
   Node(int value) {
      data = value;
      next = nullptr;
   }
};

// LinkedList class definition
class LinkedList {
   Node* head;   // Pointer to the first node

public:
   // Constructor to initialize the linked list
   LinkedList() {
      head = nullptr;
   }

   // Function to create the linked list from user input
   void create() {
      Node* newNode;
```
```

```
    Node* temp;
    char choice;

    do {
        int value;
        cout << "Enter the number to be stored: ";
        cin >> value;
        newNode = new Node(value);

        if (head == nullptr) {
            head = newNode;
        } else {
            temp = head;
            while (temp->next != nullptr)
                temp = temp->next;
            temp->next = newNode;
        }

        cout << "Do you have more data (y/n)? : ";
        cin >> choice;

    } while (choice == 'y' || choice == 'Y');
}

// Function to remove duplicates from the unsorted linked list
void removeDuplicates() {
    Node* ptr1 = head;
    Node* ptr2 = nullptr;
    Node* dup = nullptr;

    // Pick elements one by one and compare with all other elements
    while (ptr1 != nullptr && ptr1->next != nullptr) {
        ptr2 = ptr1;
        while (ptr2->next != nullptr) {
            if (ptr1->data == ptr2->next->data) {
                dup = ptr2->next;
                ptr2->next = ptr2->next->next;
                delete dup;
            } else {
                ptr2 = ptr2->next;
            }
        }
        ptr1 = ptr1->next;
    }
}

// Function to display the linked list
void display() {
    Node* temp = head;
    cout << "The stored numbers are: ";
    while (temp != nullptr) {
        cout << temp->data << '\t';
        temp = temp->next;
    }
    cout << endl;
}
```

```
};

int main() {
    LinkedList list;

    // Create the linked list from user input
    list.create();

    // Display the original list
    list.display();

    // Remove duplicates from the list
    list.removeDuplicates();

    // Display the list after removing duplicates
    list.display();

    return 0;
}
```

30. Swap nodes in a linked list without swapping data
C++ Code Implementation

```
#include <iostream>
using namespace std;

// Class for Node
class Node {
public:
    int data;
    Node* next;

    // Constructor
    Node(int val) : data(val), next(nullptr) {}
};

// Class for Linked List
class LinkedList {
private:
    Node* head;

public:
    LinkedList() : head(nullptr) {}

    // Function to add a node to the linked list
    void append(int val) {
        Node* newNode = new Node(val);
        if (!head) {
            head = newNode;
            return;
        }
        Node* temp = head;
        while (temp->next) {
```

```
        temp = temp->next;
    }
    temp->next = newNode;
}

// Function to print the linked list
void printList() {
    Node* temp = head;
    while (temp) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

// Function to swap two nodes without swapping data
void swapNodes(int x, int y) {
    // If x and y are the same, no need to swap
    if (x == y) return;

    Node* prevX = nullptr, * currX = head;
    while (currX && currX->data != x) {
        prevX = currX;
        currX = currX->next;
    }

    Node* prevY = nullptr, * currY = head;
    while (currY && currY->data != y) {
        prevY = currY;
        currY = currY->next;
    }

    // If either x or y is not present
    if (!currX || !currY) return;

    // If x is not the head node
    if (prevX) {
        prevX->next = currY;
    } else { // make y the new head
        head = currY;
    }

    // If y is not the head node
    if (prevY) {
        prevY->next = currX;
    } else { // make x the new head
        head = currX;
    }

    // Swap next pointers
    Node* temp = currY->next;
    currY->next = currX->next;
    currX->next = temp;
}
};
```

```
int main() {
    LinkedList list;
    list.append(1);
    list.append(2);
    list.append(3);
    list.append(4);
    list.append(5);

    cout << "Original Linked List:" << endl;
    list.printList();

    int x = 2, y = 4;
    list.swapNodes(x, y);

    cout << "Linked List after swapping " << x << " and " << y << ":" << endl;
    list.printList();

    return 0;
}
```

Explanation of the Code

1. **Node Class**:
   - Represents each node in the linked list with an integer `data` and a pointer to the next node (`next`).

2. **LinkedList Class**:
   - Contains a pointer to the head of the list (`head`).
   - The `append()` method adds new nodes to the end of the list.
   - The `printList()` method displays the contents of the linked list.

3. **swapNodes() Function**:
   - Takes two integers `x` and `y` as inputs, which represent the values of the nodes to be swapped.
   - Searches for the nodes with the given values and keeps track of their previous nodes.
   - If both nodes are found, it adjusts the pointers to swap the nodes without swapping the data.
   - Handles cases where either `x` or `y` is the head of the list.

4. **Main Function**:
   - Creates an instance of `LinkedList` and appends several nodes to it.
   - Prints the original linked list.
   - Calls the `swapNodes()` method to swap the nodes with values `2` and `4`.
   - Prints the modified linked list after the swap.

Output
When you run the code, the output will look like this:

```
Original Linked List:
1 -> 2 -> 3 -> 4 -> 5 -> NULL
Linked List after swapping 2 and 4:
1 -> 4 -> 3 -> 2 -> 5 -> NULL
```

This shows that the nodes containing the values `2` and `4` have been swapped without changing their data. The implementation correctly handles pointers, maintaining the structure of the linked list.

31. Pairwise swap elements of a given linked list :

Example 1:
Input: Linked list: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> NULL

Output: 2->1->4->3->6->5->NULL

Explanation: While traversing the linked list every time we encounter a pair we have to swap them. i.e pair(1,2) is swapped with (2,1), pair(3,4) is swapped with (4,3), and so on.

Example 2:
Input: Linked list: 1->2->3->4->5->NULL

Output: 2->1->4->3->5->NULL

Explanation: While traversing the linked list every time we encounter a pair we have to swap them. Here the last element 5 at the end of the list does not have any pair to swap it with, so it remains unchanged.

```
#include <bits/stdc++.h>

using namespace std;

class Node {
  public:
    int data;
  Node * next;
};

/* Function to pairwise swap elements */
void pairWiseSwap(Node * head) {
  Node * temp = head;

  /* Traverse further only if
  there are at-least two nodes left */
  while (temp != NULL && temp -> next != NULL) {
    /* Swap data of node with
       its next node's data */
    swap(temp -> data,
      temp -> next -> data);

    /* Move temp by 2 for the next pair */
    temp = temp -> next -> next;
  }
}

void push(Node ** head_ref, int new_data) {

  Node * new_node = new Node();

new_node -> data = new_data;
```

```
  new_node -> next = ( * head_ref);

  ( * head_ref) = new_node;
}

void printList(Node * node) {
  while (node != NULL) {
    cout << node -> data << " ";
    node = node -> next;
  }
}

int main() {
  Node * start = NULL;

  push( & start, 5);
  push( & start, 4);
  push( & start, 3);
  push( & start, 2);
  push( & start, 1);

  cout << "Linked list " << "before calling pairWiseSwap()\n";
  printList(start);

  pairWiseSwap(start);

  cout << "\nLinked list " <<
 "after calling pairWiseSwap()\n";
  printList(start);

  return 0;
}
```

Time Complexity: O(n)

Reason: Linked list traversal takes linear time complexity

Space Complexity: O(1)

32. Move last element to front of a given Linked List :

```
// C++ Program to move last element
// to front in a given linked list

#include <iostream>
using namespace std;

class Node {
  public:
    int data;
    Node *next;
    Node(int x) {
        data = x;
        next = nullptr;
    }
```

```
};

// Function to move the last node to the
// front of the linked list
Node *moveToFront(Node *head) {

    // If the list is empty or has only one node,
    // no need to move
    if (head == NULL || head->next == NULL) {
        return head;
    }

    // To keep track of the second last node
    Node *secLast = NULL;

    // To traverse to the last node
    Node *last = head;

    // Traverse the list to find the last and
    // second last nodes
    while (last->next != NULL) {
        secLast = last;
        last = last->next;
    }

    // Change the next of second last node to NULL
    secLast->next = NULL;

    // Make the last node as the new head
    last->next = head;
    head = last;

    return head;
}

void printList(Node *node) {
    while (node != NULL) {
        cout << node->data << " ";
        node = node->next;
    }
    cout << endl;
}

int main() {

    // Create a linked list 1->2->3->4->5
    Node *head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
    head->next->next->next->next = new Node(5);

    cout << "Linked list before: " << endl;
    printList(head);

    head = moveToFront(head);
```

```
    cout << "Linked list after: " << endl;
    printList(head);

    return 0;
}
```

OUTPUT :

Linked list before:
1 2 3 4 5
Linked list after:
5 1 2 3 4

Time Complexity: O(n), As we need to traverse the list once.
Auxiliary Space: O(1)

33. Intersection of two Sorted Linked Lists :

Given the head of two sorted linked lists, the task is to create a new linked list that represents the intersection of the two original lists. The new linked list should be created without modifying the original lists.

Note: The elements in the linked lists are not necessarily distinct.

Input: head1 = 1->2->3->4->6, head2 = 2->4->6->8
Output: 2 4 6
Explanation: For the given two linked list, 2, 4 and 6 are the elements in the intersection.

Input: head1 = 10->20->40->50, head2 = 15->40
Output: 40

Using Nested loop – O(m*n) time and O(min(m, n)) space :

The very basic idea is to use two nested loop. The outer loop iterates through the first linked list, picking each node one by one. For each node from the first list, the inner loop is used to search for the same value in the second linked list. If a match is found, the current node's value is added to the result list. To avoid duplicates, a data structure (such as a set or a boolean array) is used to keep track of the visited nodes in the second list. This will make sure that a node is added to the result list only once. Once all the nodes in the first list have been processed, the resulting list containing the intersection of the two input lists is returned.

Time Complexity: O(m*n), where m and n are the lengths of the two input linked lists
Auxiliary Space: O(min(m, n)), due to storing the intersect linked list

Since the two linked lists are already sorted, we can efficiently find the elements that are common to both lists. We can do this by comparing the values of the nodes in the two lists as we go through them.

By comparing the node values, we can create a new linked list that contains only the elements that are present in both of the original lists. The main idea to doing this efficiently is to go through the two lists at the same time. During iteration, we can skip those elements that don't match between the two lists, without needing to do unnecessary comparisons.

Code :

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
   int data;
   Node* next;
   Node(int val)
   {
      data = val;
      next = NULL;
   }
};

Node* findIntersection(Node* head1, Node* head2)
{
   Node *p1 = head1, *p2 = head2;
   Node *head = NULL, *tail = NULL;

   // Traverse both linked lists and find the common nodes
   while (p1 && p2) {
      if (p1->data > p2->data) {

         // Nodes don't match, move to the next node in
         // the list with the smaller value
         p2 = p2->next;
      }
      else if (p2->data > p1->data) {

         // Nodes don't match, move to the next node in
         // the list with the smaller value
         p1 = p1->next;
      }
      else {

         // Nodes match, add the common node to the
         // intersection list
         if (head == NULL) {
            head = tail = new Node(p1->data);

            // Create a new head for the intersection
            // list
         }
         else {

            // Append the new node to the end of the
            // intersection list
            tail->next = new Node(p1->data);
            tail = tail->next;
         }

         // Move to the next nodes in both lists
         p1 = p1->next;
         p2 = p2->next;
      }
   }
```

```
    return head;
}

// Driver code
int main()
{
    // Create the first linked list
    Node* head1 = new Node(1);
    head1->next = new Node(2);
    head1->next->next = new Node(3);
    head1->next->next->next = new Node(4);
    head1->next->next->next->next = new Node(6);

    // Create the second linked list
    Node* head2 = new Node(2);
    head2->next = new Node(4);
    head2->next->next = new Node(6);
    head2->next->next->next = new Node(8);

    // Find the intersection of the two linked lists
    Node* result = findIntersection(head1, head2);

    // Print the intersection list
    while (result) {
        cout << result->data << " ";
        result = result->next;
    }
    cout << endl;

    return 0;
}
```

34. Intersection point of two Linked Lists. :

Given two singly linked lists that merge into a single Y-shaped list. The two lists initially have distinct paths but eventually converge at a common node, forming a Y-shape, the task is to find and return the node where the two lists merge.

Code :

```
// C++ program to get intersection point of two linked list
// using two nested loops

#include <bits/stdc++.h>
using namespace std;

class Node {
  public:
    int data;
    Node *next;
    Node(int x) {
        data = x;
        next = NULL;
    }
};
```

```
/* function to get the intersection point of two linked
lists head1 and head2 */
Node *getIntersectionNode(Node *head1, Node *head2) {
   while (head2 != NULL) {
      Node *temp = head1;
      while (temp) {

         // if both Nodes are same
         if (temp == head2)
            return head2;
         temp = temp->next;
      }
      head2 = head2->next;
   }

   // intersection is not present between the lists
   return NULL;
}

int main() {

   // Create two linked lists
   // 1st list: 10 -> 15 -> 30
   // 2nd list: 3 -> 6 -> 9 -> 15 -> 30
   // 15 is the intersection point

   // creation of first list
     Node *head1 = new Node(10);
   head1->next = new Node(15);
   head1->next->next = new Node(30);

   // creation of second list
     Node *head2 = new Node(3);
     head2->next = new Node(6);
   head2->next->next = new Node(9);
   head2->next->next->next = head1->next;

   Node *intersectionPoint = getIntersectionNode(head1, head2);

   if (intersectionPoint == NULL)
      cout << " No Intersection Point \n";
   else
      cout << "Intersection Point: " << intersectionPoint->data << endl;
}
```

35. Segregate even and odd nodes in a Linked List :

Move Even and Append Remaining – O(n) Time and O(1) Space

1. Create an empty result list
2. Traverse the input list and move all even value nodes from the original list to the result list. If our original list was 17->15->8->12->10->5->4->1->7->6. Then after this step, we get original list as 17->15->5->1->7 and result list as 8->12->10->4->6
3. Append the remaining original list to the end of res and return result

Code :

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int x) {
        data = x;
        next = nullptr;
    }
};

Node* segregateEvenOdd(Node* head) {

    // Result list to hold even nodes
    Node* resStart = nullptr;
    Node* resEnd = nullptr;

    // Pointers for the original list
    Node* curr = head;
    Node* prev = nullptr;

    // Move all even nodes from original
    // to result
    while (curr != nullptr) {

        // If current node is even
        if (curr->data % 2 == 0) {

            // Remove the current even node
            // from the original list
            if (prev != nullptr) {
                prev->next = curr->next;
            } else {

                // If the even node is at the head
                head = curr->next;
            }

            // Add the current even node to the result list
            if (resStart == nullptr) {
                resStart = curr;
                resEnd = resStart;
            } else {
                resEnd->next = curr;
                resEnd = resEnd->next;
            }

            curr = curr->next;
        }

        // If the node is odd, just move to the next
        else {
            prev = curr;
```

```
            curr = curr->next;
        }
    }

    // If there are no even nodes, return
    // the original list
    if (resStart == nullptr)
        return head;

    // Append the remaining original list
    // (odd nodes) to the result list
    resEnd->next = head;

    // Return the result list (starting with even nodes)
    return resStart;
}

void printList(Node* node) {
    while (node != nullptr) {
        cout << node->data << " ";
        node = node->next;
    }
}

int main() {
    // Let us create a sample linked list as following
    // 0->1->4->6->9->10->11
    Node* head = new Node(0);
    head->next = new Node(1);
    head->next->next = new Node(4);
    head->next->next->next = new Node(6);
    head->next->next->next->next = new Node(9);
    head->next->next->next->next->next = new Node(10);
    head->next->next->next->next->next->next = new Node(11);

    cout << "Original Linked list: ";
    printList(head);

    head = segregateEvenOdd(head);

    cout << "\nModified Linked list: ";
    printList(head);

    return 0;
}
```

36. For the given array [22, 7, 2, 19, 10,18,15,13] and using the middle element as the pivot, show the sorting process with Quick Sort. :

Code :

```
#include <iostream>
using namespace std;

class QuickSort {
public:
    // Function to sort the array
```

```
    void sort(int arr[], int size) {
        quickSort(arr, 0, size - 1);
    }

    // Function to print the array
    void printArray(int arr[], int size) {
        for (int i = 0; i < size; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

private:
    // Function to partition the array
    int partition(int arr[], int low, int high) {
        int mid = low + (high - low) / 2;  // Choose the middle element as pivot
        int pivot = arr[mid];

        // Move the pivot to the end for partitioning
        swap(arr[mid], arr[high]);
        int storeIndex = low;

        for (int i = low; i < high; i++) {
            if (arr[i] < pivot) {
                swap(arr[i], arr[storeIndex]);
                storeIndex++;
            }
        }

        // Move the pivot to its final place
        swap(arr[storeIndex], arr[high]);
        return storeIndex;  // Return the index of the pivot
    }

    // Function for Quick Sort
    void quickSort(int arr[], int low, int high) {
        if (low < high) {
            int pivotIndex = partition(arr, low, high);
            quickSort(arr, low, pivotIndex - 1);  // Sort left part
            quickSort(arr, pivotIndex + 1, high); // Sort right part
        }
    }
};

int main() {
    int arr[] = {22, 7, 2, 19, 10, 18, 15, 13};
    int size = sizeof(arr) / sizeof(arr[0]);
    QuickSort qs;

    cout << "Original array: ";
    qs.printArray(arr, size);

    qs.sort(arr, size);

    cout << "Sorted array: ";
    qs.printArray(arr, size);
```

```
    return 0;
}
```

Algorithm:

1.Initial Array: [22, 7, 2, 19, 10, 18, 15, 13]

2.First Call to partition:

Pivot: Middle element = 19 (index 3).
Move 19 to the end:
[22, 7, 2, 13, 10, 18, 15, 19]

Rearrangement (less than and greater than pivot):

22: greater (no swap)
7: less (swap with storeIndex)
2: less (swap with storeIndex)
13: less (swap with storeIndex)
10: less (swap with storeIndex)
18: less (swap with storeIndex)

Final position of pivot 19: [7, 2, 13, 10, 18, 15, 19, 22]

3.Recursive Calls:

Left subarray: [7, 2, 13, 10, 18, 15]
Right subarray: [22] (already sorted)

4.Sorting Left Subarray [7, 2, 13, 10, 18, 15]:
Pivot: 10 (middle element).
Move 10 to the end: [7, 2, 13, 15, 18, 10]

Rearrangement:

- 7: less (swap with storeIndex)
- 2: less (swap with storeIndex)
- 13: greater (no swap)
- 15: greater (no swap)
- 18: greater (no swap)

Final position of pivot 10:

[2, 7, 10, 15, 18]

5.Sorting Subarrays:

Left of 10: [2, 7]
Pivot: 7
Move 7 to the end: [2, 7]
Right of 10: [15, 18] (already sorted).

6.Final Sorted array: Combining all parts:

[2, 7, 10, 15, 18, 19, 22]

37. For the given array [22, 7, 2, 19, 10,18,15,13] and using the Last element as the pivot, show the sorting process with Quick Sort.

Code :

```
#include <iostream>
using namespace std;

class QuickSort {
public:
   // Function to sort the array
   void sort(int arr[], int size) {
      quickSort(arr, 0, size - 1);
   }

   // Function to print the array
   void printArray(int arr[], int size) {
      for (int i = 0; i < size; i++) {
         cout << arr[i] << " ";
      }
      cout << endl;
   }

private:
   // Function to partition the array
   int partition(int arr[], int low, int high) {
      // Choose the last element as the pivot
      int pivot = arr[high];
      int storeIndex = low;

      for (int i = low; i < high; i++) {
         if (arr[i] < pivot) {
            swap(arr[i], arr[storeIndex]);
            storeIndex++;
         }
      }

      // Move the pivot to its final place
      swap(arr[storeIndex], arr[high]);
      return storeIndex;  // Return the index of the pivot
   }

   // Function for Quick Sort
   void quickSort(int arr[], int low, int high) {
      if (low < high) {
         int pivotIndex = partition(arr, low, high);
         quickSort(arr, low, pivotIndex - 1);  // Sort left part
         quickSort(arr, pivotIndex + 1, high); // Sort right part
      }
   }
};

int main() {
   int arr[] = {22, 7, 2, 19, 10, 18, 15, 13};
   int size = sizeof(arr) / sizeof(arr[0]);
```

```
    QuickSort qs;

    cout << "Original array: ";
    qs.printArray(arr, size);

    qs.sort(arr, size);

    cout << "Sorted array: ";
    qs.printArray(arr, size);
    return 0;
}
```

OUTPUT :

Original array: 22 7 2 19 10 18 15 13
Sorted array: 2 7 10 13 15 18 19 22

Algorithm :

1.Initial Array: [22, 7, 2, 19, 10, 18, 15, 13]

2.First Call to partition:

Pivot: Last element = 13.
Rearrangement:
22: greater (no swap)
7: less (swap with storeIndex)
2: less (swap with storeIndex)
19: greater (no swap)
10: less (swap with storeIndex)
18: greater (no swap)
Finally, swap the pivot:
Pivot index: 3.
[7, 2, 10, 13, 19, 18, 15, 22]

3.Recursive Calls:

Left subarray: [7, 2, 10]
Right subarray: [19, 18, 15, 22]

4.Sorting Left Subarray [7, 2, 10]:

Pivot: 10.

Rearrangement:
7: less (swap with storeIndex)
2: less (swap with storeIndex)
Finally, swap the pivot:

[2, 7, 10]
Pivot index: 2

5.Sorting Subarrays:

Left of 10: [2, 7]
Pivot: 7.

Rearrangement results in:

[2, 7]

Right of 10: [] (empty, already sorted).

6.Sorting Right Subarray [19, 18, 15, 22]:

Pivot: 22.
Rearrangement:
19: less (swap with storeIndex)
18: less (swap with storeIndex)
Finally, swap the pivot: [19, 18, 15, 22]

Pivot index: 3.

7.Sorting Further:

Left of 22: [19, 18, 15]
Pivot: 15.
Rearrangement: [15, 18, 19]
Right of 22: [] (empty, already sorted).

8.Final Sorted Array: Combining all parts:
[2, 7, 10, 13, 15, 18, 19, 22]

38. Given an integer array nums, move all the even integers at the beginning of the array followed by all the odd integers. Input: nums (3,1,2,4] Output: [2,4,3,1] or [4,2,3,1], [2,4,1,3], and (4,2,1,3]. :

```
#include <iostream>
using namespace std;

void rearrangeEvenOdd(int nums[], int size) {
    int left = 0, right = size - 1;

    while (left < right) {
        // Move left pointer to the right if it's even
        while (left < right && nums[left] % 2 == 0) {
            left++;
        }
        // Move right pointer to the left if it's odd
        while (left < right && nums[right] % 2 != 0) {
            right--;
        }
        // Swap if left is odd and right is even
        if (left < right) {
            swap(nums[left], nums[right]);
        }
    }
}
```

```
void printArray(int nums[], int size) {
    for (int i = 0; i < size; i++) {
        cout << nums[i] << " ";
    }
    cout << endl;
}

int main() {
    int nums[] = {3, 1, 2, 4};
    int size = sizeof(nums) / sizeof(nums[0]);

    cout << "Original array: ";
    printArray(nums, size);

    rearrangeEvenOdd(nums, size);

    cout << "Rearranged array: ";
    printArray(nums, size);

    return 0;
}
```

Explaination :

1.Function rearrangeEvenOdd:

This function uses two pointers (left and right).
The left pointer moves right until it finds an odd number.
The right pointer moves left until it finds an even number.
When both pointers find an odd and even number respectively, they swap these values.
This process continues until the pointers meet.

2.Function printArray:
A utility function to print the array elements.

3.main Function:
Initializes the array and its size.
Calls the rearranging function and prints the original and rearranged arrays.

39. Given an array nums with n objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue. We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively. Write the function for the above test case.

```
#include <iostream>
using namespace std;

void sortColors(int nums[], int size) {
    int low = 0;    // Pointer for the next position of 0 (red)
    int mid = 0;    // Pointer for the current element
    int high = size - 1; // Pointer for the next position of 2 (blue)

    while (mid <= high) {
        if (nums[mid] == 0) {
            swap(nums[low], nums[mid]);
```

```
            low++;
            mid++;
        } else if (nums[mid] == 1) {
            mid++;
        } else {
            swap(nums[mid], nums[high]);
            high--;
        }
    }
}

void printArray(int nums[], int size) {
    for (int i = 0; i < size; i++) {
        cout << nums[i] << " ";
    }
    cout << endl;
}

int main() {
    int nums[] = {2, 0, 2, 1, 1, 0};
    int size = sizeof(nums) / sizeof(nums[0]);

    cout << "Original array: ";
    printArray(nums, size);

    sortColors(nums, size);

    cout << "Sorted array: ";
    printArray(nums, size);

    return 0;
}
```

Explaination :

1.Function sortColors:

This function uses three pointers: low, mid, and high.
low tracks the position to place the next 0 (red).
mid traverses through the array.
high tracks the position to place the next 2 (blue).
The while loop continues until mid exceeds high.
If the current element (nums[mid]) is 0, it is swapped with the element at low, and both pointers (low and mid) are incremented.
If the element is 1, only mid is incremented.
If the element is 2, it is swapped with the element at high, and high is decremented (but mid is not incremented until after the swap is complete to check the new element at that position).

2.Function printArray:

Utility function to print the array elements.

3.main Function:
Initializes the array and its size.
Calls the sorting function and prints the original and sorted arrays.

40. Given an array of 10,000 integers, which sorting algorithm would you choose among Insertion Sort, Shell Sort, Radix Sort, Quick Sort, and Merge Sort to achieve the best performance and why?

For sorting an array of 10,000 integers, Quick Sort would typically be the best choice among the options provided. Here's a brief analysis of each algorithm's performance and why Quick Sort is often preferred:

Algorithm Analysis

1. Insertion Sort:
   - Time Complexity: O(n^2) in the average and worst case; O(n) in the best case (nearly sorted array).
   - Space Complexity: O(1).
   - Use Case: Efficient for small arrays or nearly sorted data, but not suitable for larger arrays like 10,000 integers due to its quadratic time complexity.

2. Shell Sort:
   - Time Complexity: Depends on the gap sequence used; generally O(n^(3/2)) or O(n log^2 n) in practice.
   - Space Complexity: O(1).
   - Use Case: Better than insertion sort for medium-sized arrays, but still not as efficient as the best comparison sorts for larger arrays.

3. Radix Sort:
   - Time Complexity: O(nk), where k is the number of digits in the largest number.
   - Space Complexity: O(n + k).
   - Use Case: Non-comparison sort that can be very efficient for integers, but it's not suitable if the range of numbers is very large compared to the number of integers (10,000 in this case). The constant factors can make it slower than comparison sorts for certain inputs.

4. Quick Sort:
   - Time Complexity: O(n log n) on average; O(n^2) in the worst case (which can be mitigated with good pivot selection strategies, such as using the median of medians).
   - Space Complexity: O(log n) for the stack space in recursive calls.
   - Use Case: Efficient for large datasets and has excellent cache performance. It's generally faster in practice compared to other O(n log n) algorithms due to its smaller constant factors.

5. Merge Sort:
   - Time Complexity: O(n log n) for all cases.
   - Space Complexity: O(n) due to the need for temporary arrays.
   - Use Case: Stable and good for linked lists, but not in-place, which can lead to higher memory usage compared to Quick Sort.

Conclusion

Given the typical performance characteristics, **Quick Sort** would be the preferred choice for sorting an array of 10,000 integers due to:

- **Average-case efficiency**: O(n log n) makes it suitable for large datasets.
- **Space efficiency**: O(log n) stack space usage is generally acceptable.
- **Practical speed**: It often outperforms other O(n log n) algorithms like Merge Sort due to better cache locality.

Caveats

- Worst-case scenario: To avoid the worst-case performance (O(n^2)), it's important to implement Quick Sort with a good pivot selection strategy, such as choosing the median of a small sample or using randomization.

In summary, Quick Sort is typically the best choice for a general-purpose sorting algorithm for an array of 10,000 integers in terms of performance and practicality.

41. Sort the following array using Quick Sort and describe the partitioning process:
Array: [9, 7, 5, 11, 12, 2, 14, 3, 10, 6] :

```
#include <iostream>
using namespace std;

class QuickSort {
public:
    void sort(int arr[], int size) {
        quickSort(arr, 0, size - 1);
    }

    void printArray(int arr[], int size) {
        for (int i = 0; i < size; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

private:
    int partition(int arr[], int low, int high) {
        int pivot = arr[high]; // Choose the last element as the pivot
        int i = low - 1;       // Pointer for the smaller element

        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++; // Increment the index of smaller element
                swap(arr[i], arr[j]); // Swap
            }
        }
        swap(arr[i + 1], arr[high]); // Move the pivot to its correct position
        return i + 1; // Return the partition index
    }

    void quickSort(int arr[], int low, int high) {
        if (low < high) {
            int pivotIndex = partition(arr, low, high);
            quickSort(arr, low, pivotIndex - 1);  // Sort the left part
            quickSort(arr, pivotIndex + 1, high); // Sort the right part
        }
    }
};

int main() {
    int arr[] = {9, 7, 5, 11, 12, 2, 14, 3, 10, 6};
    int size = sizeof(arr) / sizeof(arr[0]);
    QuickSort qs;

    cout << "Original array: ";
    qs.printArray(arr, size);
```

```
    qs.sort(arr, size);

    cout << "Sorted array: ";
    qs.printArray(arr, size);

    return 0;
}
```

Partitioning Process :

Partitioning Process
Let's describe the partitioning process step-by-step:

1.Initial Array:[9, 7, 5, 11, 12, 2, 14, 3, 10, 6]

2.First Call to partition:

Pivot: 6 (the last element).
Process:
Start with i = -1.
Traverse from j = 0 to j = 8:
9 (greater than 6): no action.
7 (greater than 6): no action.
5 (less than 6): increment i to 0, swap 5 with 9: [5, 7, 9, 11, 12, 2, 14, 3, 10, 6]
11 (greater than 6): no action.
12 (greater than 6): no action.
2 (less than 6): increment i to 1, swap 2 with 7: [5, 2, 9, 11, 12, 7, 14, 3, 10, 6]
14 (greater than 6): no action.
3 (less than 6): increment i to 2, swap 3 with 9: [5, 2, 3, 11, 12, 7, 14, 9, 10, 6]
10 (greater than 6): no action.
Final Swap: Swap pivot 6 with 11: [5, 2, 3, 6, 12, 7, 14, 9, 10, 11]
Pivot Index: 3.

3.Recursive Calls:

Left Subarray: [5, 2, 3] (low = 0, high = 2).
Right Subarray: [12, 7, 14, 9, 10, 11] (low = 4, high = 9).

4.Sorting Left Subarray [5, 2, 3]:

Pivot: 3.
Process:
Start with i = -1.
Traverse from j = 0 to j = 1:
5 (greater than 3): no action.
2 (less than 3): increment i to 0, swap 2 with 5: [2, 5, 3]
Final Swap: Swap 3 with 5: [2, 3, 5]

5.Sorting Right Subarray [12, 7, 14, 9, 10, 11]:

Pivot: 11.
Process:
Start with i = 3.
Traverse from j = 4 to j = 8:
12 (greater than 11): no action.
7 (less than 11): increment i to 4, swap 7 with 12: [7, 12, 14, 9, 10, 11]
14 (greater than 11): no action.

9 (less than 11): increment i to 5, swap 9 with 12: [7, 9, 14, 12, 10, 11]
10 (less than 11): increment i to 6, swap 10 with 14: [7, 9, 10, 12, 14, 11]
Final Swap: Swap 11 with 12: [7, 9, 10, 11, 14, 12]
Final Sorted Array: Combining all parts: [2, 3, 5, 6, 7, 9, 10, 11, 12, 14]

42. Given the array [3, 6, 8, 10, 1, 2, 1], sort it using Quick Sort with the last element as the pivot.
NOTE : Same type of question as : 37. For the given array [22, 7, 2, 19, 10,18,15,13] and using the Last element as the pivot, show the sorting process with Quick Sort.

43. Given the array nums, for each nums[i] find out how many numbers in the array are smaller than it. If the Input is: nums (8,1,2,2,3] then the Output will be: [4,0,1,1,3).

Code :
// Given the array nums, for each nums[i] find out how many numbers in the array are smaller than it. If the Input is: nums [8,1,2,2,3] then the Output will be: [4,0,1,1,3].

```
#include<iostream>
using namespace std;

int main(){
    int n;

    // Ask the user to enter the size of the array
    cout << "Enter the number of elements: ";
    cin >> n;

    // Dynamically allocate arrays using new
    int* nums = new int[n];     // Allocate memory for the nums array
    int* count = new int[n];    // Allocate memory for the count array

    // Initialize count array to zero
    for(int i = 0; i < n; i++) {
        count[i] = 0;
    }

    // Input elements into the nums array
    for(int i = 0; i < n; i++) {
        cout << "Enter element " << i + 1 << ": ";
        cin >> nums[i];
    }

    // Count how many numbers are smaller than the current number
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            if(nums[i] > nums[j]) {
                count[i]++;
            }
        }
    }

    // Output the result
    cout << "Count array is: ";
    for(int i = 0; i < n; i++) {
        cout << count[i] << " ";
    }
```

```
    cout << endl;


    delete[] nums;
    delete[] count;

    return 0;
}
```

//----------

44. Given an integer array nums, move all the even integers at the beginning of the array followed by all the odd integers.
NOTE : Similar to Q35.Segregate even and odd nodes in a Linked List :

Code :

```
// Given an integer array nums, move all the even integers at the beginning of the array followed by all the odd integers.

#include<iostream>
using namespace std;

int main(){
    int n;

    // Ask the user to enter the size of the array
    cout << "Enter the number of elements: ";
    cin >> n;
    int count = 0;
    // Dynamically allocate arrays using new
    int* nums = new int[n];     // Allocate memory for the nums array
    int* newarr = new int[n];    // Allocate memory for the count array

    // Initialize count array to zero
    for(int i = 0; i < n; i++) {
        newarr[i] = 0;
    }

    // Input elements into the nums array
    for(int i = 0; i < n; i++) {
        cout << "Enter element " << i + 1 << ": ";
        cin >> nums[i];
    }

    for(int i = 0; i < n; i++) {
        if(nums[i]%2 == 0){
            newarr[count] = nums[i];
            count++;
        }
    }

    for(int i = 0; i < n; i++) {
        if(nums[i]%2 != 0){
```

```
            newarr[count] = nums[i];
            count++;
        }
    }

    for(int i = 0; i < n; i++) {
        nums[i] = newarr[i];
        cout<<nums[i]<<" ";
    }
    return 0;
}



//-----------------------------------------------------------
```

45. Given an array of 2n integers, your task is to group these integers into n pairs of integer, say (al, bl), (a2, 62),...., (an, bn) which makes sum of min(ai, bi) for all i from 1 to n as large as possible.

```
#include <iostream>
#include <algorithm>

class Solution {
public:
    int arrayPairSum(int nums[], int size) {
        std::sort(nums, nums + size);
        int result = 0;
        for (int i = 0; i < size - 1; i += 2) {
            result += nums[i];
        }
        return result;
    }
};

int main() {
    int nums[] = {1, 4, 3, 2}; // Example array
    int size = sizeof(nums) / sizeof(nums[0]);
    Solution sol;
    int result = sol.arrayPairSum(nums, size);
    std::cout << "Result: " << result << std::endl;
    return 0;
}
```

Explanation :
Size Calculation: The size of the array is determined by sizeof(nums) / sizeof(nums[0]).
Sorting: The array is sorted using std::sort(nums, nums + size).
Summation Loop: The loop iterates over the array, adding the elements at even indices.

Time Complexity: O(NlogN), N is length of the input array
Space Complexity: O(1)

46. Given an integer array nums, return true if any value appears at least twice in the array, and return false if every element is distinct.
CODE :

```
#include<iostream>
using namespace std;

bool result(int count [],int n){
    for(int i = 0; i < n; i++) {
        if(count[i]>0){
            return true;
        }
        else return false;
    }
}
int main(){
    int n;

    // Ask the user to enter the size of the array
    cout << "Enter the number of elements: ";
    cin >> n;

    // Dynamically allocate arrays using new
    int* nums = new int[n];     // Allocate memory for the nums array
    int* count = new int[n];    // Allocate memory for the count array

    // Initialize count array to zero
    for(int i = 0; i < n; i++) {
        count[i] = 0;
    }

    // Input elements into the nums array
    for(int i = 0; i < n; i++) {
        cout << "Enter element " << i + 1 << ": ";
        cin >> nums[i];
    }

    for(int i = 0; i < n; i++) {
        for(int j = i+1; j < n; j++) {
            if(nums[i] == nums[j]){
                count[i]++;
            }
        }
    }

    if(result(count,n)){
        cout<<"True";
    }
    else{
        cout<<"False";
    }
    return 0;
}
```

47. Given an array nums containing distinct numbers in the range [0, n], return the only number in the range that is missing from the array.
CODE :

```
#include<iostream>
using namespace std;

int main(){
   int n,sum=0;

   // Ask the user to enter the size of the array
   cout << "Enter the number of elements: ";
   cin >> n;

   // Dynamically allocate arrays using new
   int* nums = new int[n];     // Allocate memory for the nums array
   int* count = new int[n];    // Allocate memory for the count array

   // Initialize count array to zero
   for(int i = 0; i < n; i++) {
      count[i] = 0;
   }

   // Input elements into the nums array
   for(int i = 0; i < n; i++) {
      cout << "Enter element " << i + 1 << ": ";
      cin >> nums[i];
      sum = sum + nums[i];
   }

   cout<<(n*(n+1)/2)-sum;
   return 0;
}
```

48. You are given an integer array score of size, where score[i] is the score of the ith athlete in a competition. All the scores are guaranteed to be unique. The athletes are placed based on their scores, where the 1st place athlete has the highest score, the 2nd place athlete has the 2nd highest score, and so on. The placement of each athlete determines their rank: The 1st place athlete's rant is "Gold Medal". The 2nd place athlete's rank is "Silver Medal". The 3rd place athlete's rank is "Bronze Medal". For the 4th place to the nth place athlete, their rank is their placement number (i.e., the xth place athlete's rank is "x").Return an array answer of size n where answer[i] is the rank of the ith athlete.

```
#include <iostream>
#include <string>
using namespace std;

// Class to represent an Athlete
class Athlete {
public:
   int score;
   int index; // To keep track of the original index
   Athlete(){}
   Athlete(int s, int i){
      score = s;
      index = i;
   }
};

// Function to partition the array for Quick Sort
```

```
int partition(Athlete arr[], int low, int high) {
    int pivot = arr[high].score;
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j].score > pivot) { // Sort in descending order
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

// Quick Sort function to sort athletes by their scores
void quickSort(Athlete arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Function to assign ranks based on sorted scores
void assignRanks(Athlete athletes[], int n, string ranks[]) {
    for (int i = 0; i < n; i++) {
        if (i == 0) {
            ranks[athletes[i].index] = "Gold Medal";
        } else if (i == 1) {
            ranks[athletes[i].index] = "Silver Medal";
        } else if (i == 2) {
            ranks[athletes[i].index] = "Bronze Medal";
        } else {
            ranks[athletes[i].index] = to_string(i + 1); // For 4th place and beyond
        }
    }
}

// Function to display the ranks
void displayRanks(string ranks[], int size) {
    cout << "\nAthlete Ranks:" << endl;
    for (int i = 0; i < size; i++) {
        cout << "Athlete " << i + 1 << ": " << ranks[i] << endl;
    }
}

// Main function
int main() {
    const int MAX_ATHLETES = 100; // Maximum number of athletes
    Athlete athletes[MAX_ATHLETES];
    string ranks[MAX_ATHLETES];
    int size = 0;
    int choice;

    do {
        cout << "\nAthlete Ranking System Menu:" << endl;
```

```
        cout << "1. Add Athlete" << endl;
        cout << "2. Display Ranks" << endl;
        cout << "3. Exit" << endl;
        cout << "Enter your choice (1-3): ";
        cin >> choice;

        switch (choice) {
            case 1: {
                if (size < MAX_ATHLETES) {
                    int score;
                    cout << "Enter the score of athlete " << size + 1 << ": ";
                    cin >> score;
                    athletes[size] = Athlete(score, size); // Store score and original index
                    size++;
                    cout << "Athlete added successfully!" << endl;
                } else {
                    cout << "Maximum number of athletes reached." << endl;
                }
                break;
            }
            case 2:
                if (size > 0) {
                    // Sort athletes based on scores
                    quickSort(athletes, 0, size - 1);
                    // Assign ranks based on sorted scores
                    assignRanks(athletes, size, ranks);
                    // Display the ranks
                    displayRanks(ranks, size);
                } else {
                    cout << "No athletes to display ranks." << endl;
                }
                break;
            case 3:
                cout << "Exiting the program." << endl;
                break;
            default:
                cout << "Invalid choice. Please select again." << endl;
        }
    } while (choice != 3);

    return 0;
}
```

49. A library needs to manage its book catalog efficiently. Books should be sorted by their titles in alphabetical order to make it easier for librarians and patrons to locate them.

```
#include <iostream>
#include <string>
using namespace std;

// Class to represent a Book
class Book
{
public:
    string title;
```

```
    string author;
    int publicationYear;
    Book(){}
    // Constructor to initialize a Book
    Book(string t, string a, int year){
        title = t;
        author = a;
        publicationYear = year;
    }

    // Method to display the book details
    void displayBook()
    {
        cout << "Title: " << title << ", Author: " << author
            << ", Year: " << publicationYear << endl;
    }
};

// Merge function for Merge Sort
void merge(Book arr[], int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;

    Book *L = new Book[n1];
    Book *R = new Book[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2)
    {
        if (L[i].title <= R[j].title)
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1)
    {
        arr[k] = L[i];
        i++;
        k++;
    }
```

```
    while (j < n2)
    {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}

// Merge Sort function to sort books by title
void mergeSort(Book arr[], int left, int right)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

// Function to display all books
void displayBooks(Book books[], int size)
{
    if (size == 0)
    {
        cout << "No books in the catalog." << endl;
    }
    else
    {
        for (int i = 0; i < size; i++)
        {
            books[i].displayBook();
        }
    }
}

// Function to add a book to the catalog
void addBook(Book books[], int &size) {
    string title, author;
    int publicationYear;

    cout << "Enter book title: ";
    cin >> title; // Using cin instead of getline
    cout << "Enter author: ";
    cin >> author; // Using cin instead of getline
    cout << "Enter publication year: ";
    cin >> publicationYear;

    books[size] = Book(title, author, publicationYear);
    size++;
    cout << "Book added successfully!" << endl;
}
```

```
// Main function
int main()
{
    const int MAX_BOOKS = 100; // Maximum number of books
    Book books[MAX_BOOKS];
    int size = 0;
    int choice;

    do
    {
        cout << "\nLibrary Management System Menu:" << endl;
        cout << "1. Add Book" << endl;
        cout << "2. Display Books" << endl;
        cout << "3. Sort Books by Title" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice (1-4): ";
        cin >> choice;

        switch (choice)
        {
        case 1:
            addBook(books, size);
            break;
        case 2:
            displayBooks(books, size);
            break;
        case 3:
            mergeSort(books, 0, size - 1);
            cout << "Books sorted by title." << endl;
            break;
        case 4:
            cout << "Exiting the program." << endl;
            break;
        default:
            cout << "Invalid choice. Please select again." << endl;
        }
    } while (choice != 4);

    return 0;
}
```

50. An e-commerce website needs to display products in various sorted orders, such as by price, customer ratings, or newest arrivals. The sorting feature must be fast and responsive to user selections.

```
#include <iostream>
#include <string>
using namespace std;

// Class to represent a Product
class Product {
public:
    string name;
```

```
    string category;
    int price;          // Price in cents
    double rating;      // Customer rating out of 5

    Product(){}
    Product(string n, string c, int p, double r){
        name = n;
        category = c;
        price = p;
        rating = r;
    }

    // Display the product details
    void displayProduct() {
        cout << "Name: " << name << ", Category: " << category
             << ", Price: $" << price / 100.0 << ", Rating: " << rating << endl;
    }
};

// Function to partition the array for Quick Sort
int partition(Product arr[], int low, int high, bool sortByPrice) {
    // Determine the pivot based on whether we are sorting by price or rating
    int pivot = sortByPrice ? arr[high].price : static_cast<int>(arr[high].rating * 100);
    int i = low - 1;

    for (int j = low; j < high; j++) {
        int currentValue = sortByPrice ? arr[j].price : static_cast<int>(arr[j].rating * 100);
        if (currentValue < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

// Quick Sort function
void quickSort(Product arr[], int low, int high, bool sortByPrice) {
    if (low < high) {
        int pi = partition(arr, low, high, sortByPrice);
        quickSort(arr, low, pi - 1, sortByPrice);
        quickSort(arr, pi + 1, high, sortByPrice);
    }
}

// Merge function for Merge Sort (for names and categories)
void merge(Product arr[], int left, int mid, int right, bool sortByCategory) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    Product* L = new Product[n1];
    Product* R = new Product[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (sortByCategory) {
            if (L[i].category <= R[j].category) {
                arr[k] = L[i];
                i++;
            } else {
                arr[k] = R[j];
                j++;
            }
        } else {
            if (L[i].name <= R[j].name) {
                arr[k] = L[i];
                i++;
            } else {
                arr[k] = R[j];
                j++;
            }
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}

void mergeSort(Product arr[], int left, int right, bool sortByCategory) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid, sortByCategory);
        mergeSort(arr, mid + 1, right, sortByCategory);
        merge(arr, left, mid, right, sortByCategory);
    }
}

// Function to display all products
void displayProducts(Product products[], int size) {
    for (int i = 0; i < size; i++) {
        products[i].displayProduct();
    }
}

// Main function
```

```
int main() {
    // Example products
    Product products[] = {
        Product("Laptop", "Electronics", 99999, 4.5),
        Product("Coffee Maker", "Home Appliances", 24999, 4.0),
        Product("Smartphone", "Electronics", 59999, 4.8),
        Product("Blender", "Home Appliances", 19999, 4.2),
        Product("Watch", "Fashion", 29999, 4.6)
    };

    int size = sizeof(products) / sizeof(products[0]);

    cout << "Original Products List:" << endl;
    displayProducts(products, size);

    // Sort by price using Quick Sort
    quickSort(products, 0, size - 1, true);
    cout << "\nProducts Sorted by Price (Quick Sort):" << endl;
    displayProducts(products, size);

    // Sort by names using Merge Sort
    mergeSort(products, 0, size - 1, false);
    cout << "\nProducts Sorted by Names (Merge Sort):" << endl;
    displayProducts(products, size);

    // Sort by categories using Merge Sort
    mergeSort(products, 0, size - 1, true);
    cout << "\nProducts Sorted by Categories (Merge Sort):" << endl;
    displayProducts(products, size);

    // Sort by ratings using Quick Sort (using the rating as an integer)
    quickSort(products, 0, size - 1, false);  // Here, we sort by rating
    cout << "\nProducts Sorted by Ratings (Quick Sort):" << endl;
    displayProducts(products, size);

    return 0;
}
```

51. A company has a list of job tasks that need to be scheduled based on their deadlines. Each task has a start time, end time, and priority. The goal is to sort tasks in such a way that the most urgent and high-priority tasks are handled first.

```
#include <iostream>
#include <string>

using namespace std;

// Class to represent a Task
class Task {
public:
    string taskName;
    string deadline;  // Deadline in format YYYY-MM-DD
    string startTime; // Start time in format HH:MM
    int priority;     // Priority from 1 (low) to 10 (high)

    Task() {}
```

```
    Task(string name, string dl, string st, int pr) : taskName(name), deadline(dl), startTime(st), priority(pr) {}

    // Display the task details
    void displayTask() {
        cout << "Task: " << taskName << ", Deadline: " << deadline << ", Start Time: " << startTime << ",
Priority: " << priority << endl;
    }
};

// Merge two sorted subarrays
void merge(Task arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    Task* L = new Task[n1];
    Task* R = new Task[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        // Sort primarily by priority, then by deadline, then by start time
        if (L[i].priority > R[j].priority ||
            (L[i].priority == R[j].priority && L[i].deadline < R[j].deadline) ||
            (L[i].priority == R[j].priority && L[i].deadline == R[j].deadline && L[i].startTime < R[j].startTime)) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}

// Merge Sort function for sorting tasks by priority, deadline, and start time
void mergeSort(Task arr[], int left, int right) {
```

```
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

// Function to display all tasks
void displayTasks(Task tasks[], int size) {
    if (size == 0) {
        cout << "No tasks scheduled.\n";
        return;
    }

    for (int i = 0; i < size; i++) {
        tasks[i].displayTask();
    }
}

// Menu function
void menu() {
    cout << "\n=== Task Management System ===\n";
    cout << "1. Add a new task\n";
    cout << "2. View all tasks\n";
    cout << "3. Sort tasks by priority, deadline, and start time (Merge Sort)\n";
    cout << "4. Exit\n";
    cout << "Enter your choice: ";
}

int main() {
    int size = 0;
    int capacity = 100;
    Task tasks[100];  // Array to store up to 100 tasks

    int choice;

    do {
        menu();
        cin >> choice;

        switch (choice) {
        case 1: {
            // Add a new task
            if (size >= capacity) {
                cout << "Task list is full!\n";
                break;
            }

            string name, deadline, startTime;
            int priority;
            cout << "Enter task name (single word): ";
            cin >> name;
            cout << "Enter deadline (YYYY-MM-DD): ";
```

```
            cin >> deadline;
            cout << "Enter start time (HH:MM): ";
            cin >> startTime;
            cout << "Enter priority (1 to 10): ";
            cin >> priority;

            tasks[size++] = Task(name, deadline, startTime, priority);
            cout << "Task added successfully!\n";
            break;
        }
        case 2: {
            // View all tasks
            if (size == 0) {
                cout << "No tasks scheduled.\n";
            } else {
                displayTasks(tasks, size);
            }
            break;
        }
        case 3: {
            // Sort tasks by priority, deadline, and start time using Merge Sort
            if (size == 0) {
                cout << "No tasks to sort.\n";
            } else {
                mergeSort(tasks, 0, size - 1);
                cout << "Tasks sorted by priority, deadline, and start time (Merge Sort).\n";
            }
            break;
        }
        case 4: {
            cout << "Exiting the program. Goodbye!\n";
            break;
        }
        default: {
            cout << "Invalid choice! Please try again.\n";
            break;
        }
        }
    } while (choice != 4);

    return 0;
}
```

52. A school needs to sort students' grades to determine rankings for awards and honor rolls. The system should sort students by their average grades and, in case of ties, by their names.

```
#include <iostream>
#include <string>

using namespace std;

// Class to represent a Student
class Student {
public:
    string name;
    double averageGrade;
```

```
    Student() {}

    Student(string n, double g){
        name = n;
        averageGrade = g;
    }

    // Display the student details
    void displayStudent() {
        cout << "Name: " << name << ", Average Grade: " << averageGrade << endl;
    }
};

// Merge two sorted subarrays
void merge(Student arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    Student* L = new Student[n1];
    Student* R = new Student[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        // Sort primarily by grade, and by name in case of ties
        if (L[i].averageGrade > R[j].averageGrade ||
            (L[i].averageGrade == R[j].averageGrade && L[i].name < R[j].name)) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}
```

```
// Merge Sort function for sorting students by grades and names
void mergeSort(Student arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

// Function to display all students
void displayStudents(Student students[], int size) {
    if (size == 0) {
        cout << "No students found.\n";
        return;
    }

    for (int i = 0; i < size; i++) {
        students[i].displayStudent();
    }
}

// Menu function
void menu() {
    cout << "\n=== Student Management System ===\n";
    cout << "1. Add a new student\n";
    cout << "2. View all students\n";
    cout << "3. Sort students by average grade (Merge Sort)\n";
    cout << "4. Exit\n";
    cout << "Enter your choice: ";
}

int main() {
    int size = 0;
    int capacity = 100;
    Student students[100];  // Array to store up to 100 students

    int choice;

    do {
        menu();
        cin >> choice;

        switch (choice) {
        case 1: {
            // Add a new student
            if (size >= capacity) {
                cout << "Student list is full!\n";
                break;
            }

            string name;
            double grade;
```

```
            cout << "Enter student name: ";
            cin >> name;
            cout << "Enter average grade: ";
            cin >> grade;

            students[size++] = Student(name, grade);
            cout << "Student added successfully!\n";
            break;
        }
        case 2: {
            // View all students
            if (size == 0) {
                cout << "No students found.\n";
            } else {
                displayStudents(students, size);
            }
            break;
        }
        case 3: {
            // Sort by average grade using Merge Sort
            if (size == 0) {
                cout << "No students to sort.\n";
            } else {
                mergeSort(students, 0, size - 1);
                cout << "Students sorted by average grade and name (Merge Sort).\n";
            }
            break;
        }
        case 4: {
            cout << "Exiting the program. Goodbye!\n";
            break;
        }
        default: {
            cout << "Invalid choice! Please try again.\n";
            break;
        }
        }
    } while (choice != 4);

    return 0;
}
```

53. An event management system needs to sort a list of scheduled events by their start times to create a daily agenda. The system must ensure that events are ordered correctly and efficiently.

```
#include <iostream>
#include <string>

using namespace std;

// Class to represent an Event
class Event {
public:
    string eventName;
    string startTime; // Time in HH:MM format
```

```
    Event() {}

    Event(string name, string time){
        eventName = name;
        startTime = time;
    }

    // Display the event details
    void displayEvent() {
        cout << "Event: " << eventName << ", Start Time: " << startTime << endl;
    }

    // Helper function to compare times in HH:MM format
    bool operator<(const Event& other) {
        return startTime < other.startTime;
    }
};

// Quick Sort for sorting events by start times
int partition(Event arr[], int low, int high) {
    Event pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j].startTime < pivot.startTime) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(Event arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Function to display all events
void displayEvents(Event events[], int size) {
    if (size == 0) {
        cout << "No events scheduled.\n";
        return;
    }

    for (int i = 0; i < size; i++) {
        events[i].displayEvent();
    }
}

// Menu function
void menu() {
```

```
    cout << "\n=== Event Management System ===\n";
    cout << "1. Add a new event\n";
    cout << "2. View all events\n";
    cout << "3. Sort events by start time (Quick Sort)\n";
    cout << "4. Exit\n";
    cout << "Enter your choice: ";
}

int main() {
    int size = 0;
    int capacity = 100;
    Event events[100];  // Array to store up to 100 events

    int choice;

    do {
        menu();
        cin >> choice;

        switch (choice) {
        case 1: {
            // Add a new event
            if (size >= capacity) {
                cout << "Event list is full!\n";
                break;
            }

            string name, startTime;
            cout << "Enter event name (single word): ";
            cin >> name;
            cout << "Enter start time (HH:MM): ";
            cin >> startTime;

            events[size++] = Event(name, startTime);
            cout << "Event added successfully!\n";
            break;
        }
        case 2: {
            // View all events
            if (size == 0) {
                cout << "No events scheduled.\n";
            } else {
                displayEvents(events, size);
            }
            break;
        }
        case 3: {
            // Sort by start time using Quick Sort
            if (size == 0) {
                cout << "No events to sort.\n";
            } else {
                quickSort(events, 0, size - 1);
                cout << "Events sorted by start time (Quick Sort).\n";
            }
            break;
        }
```

```
        case 4: {
            cout << "Exiting the program. Goodbye!\n";
            break;
        }
        default: {
            cout << "Invalid choice! Please try again.\n";
            break;
        }
        }
    } while (choice != 4);

    return 0;
}
```

54. A contact management app needs to sort contacts by names, phone numbers, or email addresses. Users should be able to quickly locate contacts based on these fields.

```
#include <iostream>

#include <string>

using namespace std;



// Class to represent a Contact

class Contact {

public:

    string name;

    string phoneNumber;

    string email;

    Contact(){

        name = "none";

        phoneNumber = "0";

        email = "none";

    }

    Contact(string n, string p, string e){

        name = n;

        phoneNumber = p;

        email = e;
```

```
    }

    // Display the contact details
    void displayContact() {
        cout << "Name: " << name << ", Phone: " << phoneNumber << ", Email: " << email << endl;
    }
};
// Quick Sort to sort contacts by phone number
int partition(Contact arr[], int low, int high) {
    string pivot = arr[high].phoneNumber;
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j].phoneNumber < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}
void quickSort(Contact arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
// Merge function for Merge Sort (for names and emails)
```

```
void merge(Contact arr[], int left, int mid, int right, bool sortByEmail) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    // Temporary arrays
    Contact* L = new Contact[n1];
    Contact* R = new Contact[n2];
    // Copy data to temporary arrays
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = left;
    // Merging the arrays
    while (i < n1 && j < n2) {
        if (sortByEmail) {
            if (L[i].email <= R[j].email) {
                arr[k] = L[i];
                i++;
            } else {
                arr[k] = R[j];
                j++;
            }
        } else {
            if (L[i].name <= R[j].name) {
                arr[k] = L[i];
                i++;
            } else {
                arr[k] = R[j];
                j++;
            }
```

```
        }
        k++;
    }
    // Copy remaining elements of L[]
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    // Copy remaining elements of R[]
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
    delete[] L;
    delete[] R;
}
void mergeSort(Contact arr[], int left, int right, bool sortByEmail) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid, sortByEmail);
        mergeSort(arr, mid + 1, right, sortByEmail);
        merge(arr, left, mid, right, sortByEmail);
    }
}
// Function to display all contacts
void displayContacts(Contact contacts[], int size) {
```

```
    for (int i = 0; i < size; i++) {
        contacts[i].displayContact();
    }
}

// Menu function
void menu() {
    cout << "\n=== Contact Management System ===\n";
    cout << "1. Add a new contact\n";
    cout << "2. View all contacts\n";
    cout << "3. Sort contacts by phone number (Quick Sort)\n";
    cout << "4. Sort contacts by name (Merge Sort)\n";
    cout << "5. Sort contacts by email (Merge Sort)\n";
    cout << "6. Exit\n";
    cout << "Enter your choice: ";
}

int main() {
    int size = 0;
    int capacity = 100;
    Contact contacts[100];  // Array to store up to 100 contacts
    int choice;
    do {
        menu();
        cin >> choice;
        switch (choice) {
        case 1: {
            // Add a new contact
```

```
    if (size >= capacity) {
        cout << "Contact list is full!\n";
        break;
    }
    string name, phone, email;
    cout << "Enter name: ";
    cin.ignore();
    getline(cin, name);
    cout << "Enter phone number: ";
    getline(cin, phone);
    cout << "Enter email: ";
    getline(cin, email);

    contacts[size++] = Contact(name, phone, email);
    cout << "Contact added successfully!\n";
    break;
}
case 2: {
    // View all contacts
    if (size == 0) {
        cout << "No contacts available.\n";
    } else {
        displayContacts(contacts, size);
    }
    break;
}
case 3: {
    // Sort by phone number using Quick Sort
```

```
    if (size == 0) {
        cout << "No contacts to sort.\n";
    } else {
        quickSort(contacts, 0, size - 1);
        cout << "Contacts sorted by phone number (Quick Sort).\n";
    }
    break;
}
case 4: {
    // Sort by name using Merge Sort
    if (size == 0) {
        cout << "No contacts to sort.\n";
    } else {
        mergeSort(contacts, 0, size - 1, false);
        cout << "Contacts sorted by name (Merge Sort).\n";
    }
    break;
}
case 5: {
    // Sort by email using Merge Sort
    if (size == 0) {
        cout << "No contacts to sort.\n";
    } else {
        mergeSort(contacts, 0, size - 1, true);
        cout << "Contacts sorted by email (Merge Sort).\n";
    }
    break;
}
```

```
        case 6: {

            cout << "Exiting the program. Goodbye!\n";

            break;

        }

        default: {

            cout << "Invalid choice! Please try again.\n";

            break;

        }

        }

    } while (choice != 6);

   return 0;

}
```

55. A hospital's patient management system needs to sort patient records by various criteria, such as appointment date, urgency level, and patient names.

CODE:

```
#include <iostream>
#include <string>
using namespace std;

// Class to represent a Patient
class Patient
{
public:
    string name;
    string appointmentDate;
    int urgencyLevel;

    Patient() {}

    Patient(string n, string date, int urgency)
    {
        name = n;
        appointmentDate = date;
        urgencyLevel = urgency;
    }

    // Display the patient details
    void displayPatient()
    {
        cout << "Name: " << name << ", Appointment Date: " << appointmentDate << ", Urgency Level: " <<
urgencyLevel << endl;
    }
```

```
};

// Quick Sort for sorting patients by urgency level
int partition(Patient arr[], int low, int high)
{
    int pivot = arr[high].urgencyLevel;
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j].urgencyLevel < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(Patient arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Merge function for Merge Sort (for names and appointment dates)
void merge(Patient arr[], int left, int mid, int right, bool sortByDate)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Temporary arrays
    Patient *L = new Patient[n1];
    Patient *R = new Patient[n2];

    // Copy data to temporary arrays
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    // Merging the arrays
    while (i < n1 && j < n2)
    {
        if (sortByDate)
        {
            if (L[i].appointmentDate <= R[j].appointmentDate)
            {
```

```
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }
    }
    else
    {
        if (L[i].name <= R[j].name)
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }
    }
    k++;
}

// Copy remaining elements of L[]
while (i < n1)
{
    arr[k] = L[i];
    i++;
    k++;
}

// Copy remaining elements of R[]
while (j < n2)
{
    arr[k] = R[j];
    j++;
    k++;
}

delete[] L;
delete[] R;
}

void mergeSort(Patient arr[], int left, int right, bool sortByDate)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid, sortByDate);
        mergeSort(arr, mid + 1, right, sortByDate);

        merge(arr, left, mid, right, sortByDate);
    }
```

```
}

// Function to display all patients
void displayPatients(Patient patients[], int size)
{
    for (int i = 0; i < size; i++)
    {
        patients[i].displayPatient();
    }
}

// Menu function
void menu()
{
    cout << "\n=== Patient Management System ===\n";
    cout << "1. Add a new patient\n";
    cout << "2. View all patients\n";
    cout << "3. Sort patients by urgency level (Quick Sort)\n";
    cout << "4. Sort patients by appointment date (Merge Sort)\n";
    cout << "5. Sort patients by name (Merge Sort)\n";
    cout << "6. Exit\n";
    cout << "Enter your choice: ";
}

int main()
{
    int size = 0;
    int capacity = 100;
    Patient patients[100]; // Array to store up to 100 patients

    int choice;

    do
    {
        menu();
        cin >> choice;

        switch (choice)
        {
        case 1:
        {
            // Add a new patient
            if (size >= capacity)
            {
                cout << "Patient list is full!\n";
                break;
            }

            string name, appointmentDate;
            int urgencyLevel;
            cout << "Enter patient name (single word): ";
            cin >> name;
            cout << "Enter appointment date (YYYY-MM-DD): ";
            cin >> appointmentDate;
            cout << "Enter urgency level (1 to 10): ";
            cin >> urgencyLevel;
```

```
    patients[size++] = Patient(name, appointmentDate, urgencyLevel);
    cout << "Patient added successfully!\n";
}
case 2:
{
    // View all patients
    if (size == 0)
    {
        cout << "No patients available.\n";
    }
    else
    {
        displayPatients(patients, size);
    }
    break;
}
case 3:
{
    // Sort by urgency level using Quick Sort
    if (size == 0)
    {
        cout << "No patients to sort.\n";
    }
    else
    {
        quickSort(patients, 0, size - 1);
        cout << "Patients sorted by urgency level (Quick Sort).\n";
    }
    break;
}
case 4:
{
    // Sort by appointment date using Merge Sort
    if (size == 0)
    {
        cout << "No patients to sort.\n";
    }
    else
    {
        mergeSort(patients, 0, size - 1, true);
        cout << "Patients sorted by appointment date (Merge Sort).\n";
    }
    break;
}
case 5:
{
    // Sort by name using Merge Sort
    if (size == 0)
    {
        cout << "No patients to sort.\n";
    }
    else
    {
        mergeSort(patients, 0, size - 1, false);
        cout << "Patients sorted by name (Merge Sort).\n";
```

```
            }
            break;
        }
        case 6:
        {
            cout << "Exiting the program. Goodbye!\n";
            break;
        }
        default:
        {
            cout << "Invalid choice! Please try again.\n";
            break;
        }
        }
    } while (choice != 6);

    return 0;
}
```

56. A supply chain system needs to sort inventory items based on various attributes, such as stock levels, reorder dates, and item categories.

```
#include <iostream>
#include <string>

using namespace std;

class InventoryItem {
public:
    string name;
    int stockLevel;
    string reorderDate;
    string category;

    InventoryItem(string n, int stock, string reorder, string cat)
        : name(n), stockLevel(stock), reorderDate(reorder), category(cat) {}

    void display() {
        cout << "Item: " << name << ", Stock Level: " << stockLevel
             << ", Reorder Date: " << reorderDate << ", Category: " << category << endl;
    }
};

class Inventory {
private:
    InventoryItem* items[100];  // Static array to hold inventory items
    int itemCount;

public:
    Inventory() : itemCount(0) {}

    void addItem(InventoryItem* item) {
        items[itemCount++] = item;
    }
```

```
    void sortByStockLevel() {
        for (int i = 0; i < itemCount - 1; i++) {
            for (int j = i + 1; j < itemCount; j++) {
                if (items[i]->stockLevel > items[j]->stockLevel) {
                    InventoryItem* temp = items[i];
                    items[i] = items[j];
                    items[j] = temp;
                }
            }
        }
    }

    void sortByReorderDate() {
        for (int i = 0; i < itemCount - 1; i++) {
            for (int j = i + 1; j < itemCount; j++) {
                if (items[i]->reorderDate > items[j]->reorderDate) {
                    InventoryItem* temp = items[i];
                    items[i] = items[j];
                    items[j] = temp;
                }
            }
        }
    }

    void sortByCategory() {
        for (int i = 0; i < itemCount - 1; i++) {
            for (int j = i + 1; j < itemCount; j++) {
                if (items[i]->category > items[j]->category) {
                    InventoryItem* temp = items[i];
                    items[i] = items[j];
                    items[j] = temp;
                }
            }
        }
    }

    void displayItems() {
        for (int i = 0; i < itemCount; i++) {
            items[i]->display();
        }
    }
};

int main() {
    Inventory inventory;

    inventory.addItem(new InventoryItem("Item1", 50, "2023-10-05", "Electronics"));
    inventory.addItem(new InventoryItem("Item2", 20, "2023-08-15", "Clothing"));
    inventory.addItem(new InventoryItem("Item3", 100, "2023-09-10", "Groceries"));

    cout << "Items sorted by Stock Level:" << endl;
    inventory.sortByStockLevel();
    inventory.displayItems();

    cout << "\nItems sorted by Reorder Date:" << endl;
    inventory.sortByReorderDate();
```

```
    inventory.displayItems();

    cout << "\nItems sorted by Category:" << endl;
    inventory.sortByCategory();
    inventory.displayItems();

    return 0;
}
```

Explanation :

InventoryItem Class:

This class holds information about individual inventory items: name, stockLevel, reorderDate, and category.
The constructor initializes these attributes.
The display() method prints the item's details.

Inventory Class:

Stores inventory items in a static array (items[100]), with a counter (itemCount) to keep track of the number of items.
The addItem() method adds a new item to the array.
Sorting functions (sortByStockLevel(), sortByReorderDate(), sortByCategory()) use nested loops to sort items based on the specified attribute.
The displayItems() method prints all the items in the inventory.

Main Function:

Creates an instance of Inventory and adds three items.
The items are sorted and displayed based on stock levels, reorder dates, and categories.

Using Linked list :

```
#include <iostream>
#include <string>

using namespace std;

class InventoryItem {
public:
    string name;
    int stockLevel;
    string reorderDate;
    string category;
    InventoryItem* next;

    InventoryItem(string n, int stock, string reorder, string cat)
        : name(n), stockLevel(stock), reorderDate(reorder), category(cat), next(nullptr) {}

    void display() {
        cout << "Item: " << name << ", Stock Level: " << stockLevel
             << ", Reorder Date: " << reorderDate << ", Category: " << category << endl;
    }
};
```

```
class Inventory {
private:
    InventoryItem* head;

public:
    Inventory() : head(nullptr) {}

    void addItem(InventoryItem* newItem) {
        if (!head) {
            head = newItem;
        } else {
            InventoryItem* temp = head;
            while (temp->next) {
                temp = temp->next;
            }
            temp->next = newItem;
        }
    }

    void sortByStockLevel() {
        InventoryItem* i = head;
        while (i) {
            InventoryItem* j = i->next;
            while (j) {
                if (i->stockLevel > j->stockLevel) {
                    swap(i->name, j->name);
                    swap(i->stockLevel, j->stockLevel);
                    swap(i->reorderDate, j->reorderDate);
                    swap(i->category, j->category);
                }
                j = j->next;
            }
            i = i->next;
        }
    }

    void sortByReorderDate() {
        InventoryItem* i = head;
        while (i) {
            InventoryItem* j = i->next;
            while (j) {
                if (i->reorderDate > j->reorderDate) {
                    swap(i->name, j->name);
                    swap(i->stockLevel, j->stockLevel);
                    swap(i->reorderDate, j->reorderDate);
                    swap(i->category, j->category);
                }
                j = j->next;
            }
            i = i->next;
        }
    }

    void sortByCategory() {
        InventoryItem* i = head;
        while (i) {
```

```
            InventoryItem* j = i->next;
            while (j) {
                if (i->category > j->category) {
                    swap(i->name, j->name);
                    swap(i->stockLevel, j->stockLevel);
                    swap(i->reorderDate, j->reorderDate);
                    swap(i->category, j->category);
                }
                j = j->next;
            }
            i = i->next;
        }
    }

    void displayItems() {
        InventoryItem* temp = head;
        while (temp) {
            temp->display();
            temp = temp->next;
        }
    }
};

int main() {
    Inventory inventory;

    inventory.addItem(new InventoryItem("Item1", 50, "2023-10-05", "Electronics"));
    inventory.addItem(new InventoryItem("Item2", 20, "2023-08-15", "Clothing"));
    inventory.addItem(new InventoryItem("Item3", 100, "2023-09-10", "Groceries"));

    cout << "Items sorted by Stock Level:" << endl;
    inventory.sortByStockLevel();
    inventory.displayItems();

    cout << "\nItems sorted by Reorder Date:" << endl;
    inventory.sortByReorderDate();
    inventory.displayItems();

    cout << "\nItems sorted by Category:" << endl;
    inventory.sortByCategory();
    inventory.displayItems();

    return 0;
}
```

Explanation :

InventoryItem Class:

Similar to the previous implementation, but now includes a next pointer to form a linked list.

Inventory Class:

Uses a linked list structure by maintaining a pointer head to the first InventoryItem.
The addItem() method inserts new items at the end of the linked list.

The sorting functions traverse the list and sort the items by swapping their attributes (name, stockLevel, reorderDate, category).
The displayItems() method prints the items by traversing the list.

Main Function:

Creates an instance of Inventory and adds items to the linked list.
Items are sorted and displayed based on stock levels, reorder dates, and categories.

57. A sports tournament needs to rank teams based on their performance metrics, such as win-loss records, points scored, and point differentials.

```cpp
#include <iostream>

using namespace std;

// Structure to represent a team in the tournament
struct Team {
    int teamID;          // Unique team identifier
    int wins;            // Number of wins
    int losses;          // Number of losses
    int pointsScored;    // Total points scored by the team
    int pointDifferential; // Point differential (points scored - points conceded)
};

// Function to swap two teams
void swapTeams(Team &a, Team &b) {
    Team temp = a;
    a = b;
    b = temp;
}

// Function to sort teams by win-loss record (more wins first)
void sortByWins(Team teams[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (teams[j].wins < teams[j+1].wins) {
                swapTeams(teams[j], teams[j+1]);
            }
        }
    }
}

// Function to sort teams by points scored (more points first)
void sortByPointsScored(Team teams[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (teams[j].pointsScored < teams[j+1].pointsScored) {
                swapTeams(teams[j], teams[j+1]);
            }
        }
    }
}

// Function to sort teams by point differential (higher differential first)
```

```
void sortByPointDifferential(Team teams[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (teams[j].pointDifferential < teams[j+1].pointDifferential) {
                swapTeams(teams[j], teams[j+1]);
            }
        }
    }
}

// Function to print the list of teams
void printTeams(Team teams[], int n) {
    for (int i = 0; i < n; i++) {
        cout << "Team ID: " << teams[i].teamID
             << ", Wins: " << teams[i].wins
             << ", Losses: " << teams[i].losses
             << ", Points Scored: " << teams[i].pointsScored
             << ", Point Differential: " << teams[i].pointDifferential << endl;
    }
}

int main() {
    // Array of teams with sample data
    Team teams[] = {
        {101, 8, 2, 400, 150},  // teamID, wins, losses, pointsScored, pointDifferential
        {102, 10, 0, 500, 200},
        {103, 7, 3, 350, 120},
        {104, 9, 1, 450, 180}
    };

    int n = sizeof(teams) / sizeof(teams[0]);

    // Sorting teams by win-loss record (more wins first)
    cout << "Teams sorted by Wins:" << endl;
    sortByWins(teams, n);
    printTeams(teams, n);
    cout << endl;

    // Sorting teams by points scored (more points first)
    cout << "Teams sorted by Points Scored:" << endl;
    sortByPointsScored(teams, n);
    printTeams(teams, n);
    cout << endl;

    // Sorting teams by point differential (higher differential first)
    cout << "Teams sorted by Point Differential:" << endl;
    sortByPointDifferential(teams, n);
    printTeams(teams, n);

    return 0;
}
```

Explanation:
1. **Structure `Team`**:
   - Contains fields: `teamID` (identifier for the team), `wins`, `losses`, `pointsScored`, and `pointDifferential`.

- This structure represents the performance metrics of each team in the tournament.

2. **`swapTeams` Function**:
   - A helper function that swaps two `Team` structures. It's used by the sorting functions to exchange team details.

3. **Sorting Functions**:
   - **`sortByWins`**: Sorts teams by the number of wins in descending order (higher win count first).
   - **`sortByPointsScored`**: Sorts teams by points scored in descending order (higher points scored first).
   - **`sortByPointDifferential`**: Sorts teams by point differential (points scored - points conceded) in descending order.
   - All three functions use the Bubble Sort algorithm for simplicity.

4. **`printTeams` Function**:
   - Prints the list of teams with their details.

5. **Main Function**:
   - Defines an array of teams with sample data.
   - Calls each sorting function (`sortByWins`, `sortByPointsScored`, `sortByPointDifferential`) and prints the sorted teams.

Key Points:
- Sorting is done in descending order for all criteria.
- The program maintains the flexibility to sort by any performance metric by simply calling the corresponding sorting function.
- We use the `swapTeams` function to rearrange the teams while preserving the order in the sorted list.

Using Linked List

```
#include <iostream>
using namespace std;

// Structure for Team node in the linked list
struct Team {
    int teamID;
    int wins;
    int losses;
    int pointsScored;
    int pointDifferential;
    Team* next;
};

// Function to append a new team node at the end of the list
void appendTeam(Team** head_ref, int id, int wins, int losses, int pointsScored, int pointDiff) {
    Team* new_team = new Team();
    new_team->teamID = id;
    new_team->wins = wins;
    new_team->losses = losses;
    new_team->pointsScored = pointsScored;
    new_team->pointDifferential = pointDiff;
    new_team->next = NULL;

    if (*head_ref == NULL) {
        *head_ref = new_team;
    } else {
```

```
        Team* temp = *head_ref;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = new_team;
    }
}

// Swap two team nodes (by swapping their data)
void swapTeams(Team* a, Team* b) {
    int tempID = a->teamID;
    int tempWins = a->wins;
    int tempLosses = a->losses;
    int tempPointsScored = a->pointsScored;
    int tempPointDiff = a->pointDifferential;

    a->teamID = b->teamID;
    a->wins = b->wins;
    a->losses = b->losses;
    a->pointsScored = b->pointsScored;
    a->pointDifferential = b->pointDifferential;

    b->teamID = tempID;
    b->wins = tempWins;
    b->losses = tempLosses;
    b->pointsScored = tempPointsScored;
    b->pointDifferential = tempPointDiff;
}

// Sort the linked list by wins
void sortByWins(Team* head) {
    Team* i;
    Team* j;
    for (i = head; i != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->wins < j->wins) {
                swapTeams(i, j);
            }
        }
    }
}

// Sort the linked list by points scored
void sortByPointsScored(Team* head) {
    Team* i;
    Team* j;
    for (i = head; i != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->pointsScored < j->pointsScored) {
                swapTeams(i, j);
            }
        }
    }
}

// Sort the linked list by point differential
void sortByPointDifferential(Team* head) {
```

```
    Team* i;
    Team* j;
    for (i = head; i != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->pointDifferential < j->pointDifferential) {
                swapTeams(i, j);
            }
        }
    }
}

// Function to print the team list
void printTeams(Team* head) {
    Team* temp = head;
    while (temp != NULL) {
        cout << "TeamID: " << temp->teamID << ", Wins: " << temp->wins
             << ", Losses: " << temp->losses << ", Points Scored: " << temp->pointsScored
             << ", Point Differential: " << temp->pointDifferential << endl;
        temp = temp->next;
    }
}

int main() {
    Team* head = NULL;

    // Adding teams to the list
    appendTeam(&head, 1, 8, 2, 400, 150);
    appendTeam(&head, 2, 10, 0, 500, 200);
    appendTeam(&head, 3, 7, 3, 350, 120);
    appendTeam(&head, 4, 9, 1, 450, 180);

    cout << "Teams sorted by Wins:\n";
    sortByWins(head);
    printTeams(head);

    cout << "\nTeams sorted by Points Scored:\n";
    sortByPointsScored(head);
    printTeams(head);

    cout << "\nTeams sorted by Point Differential:\n";
    sortByPointDifferential(head);
    printTeams(head);

    return 0;
}
```

58. A flight booking system needs to sort available flights by departure times, arrival times, and prices for users searching for flights.

```cpp
#include <iostream>

using namespace std;
```

```
// Structure to represent a flight
struct Flight {
    int flightID;       // Unique flight identifier
    int departureTime;  // Departure time in 24-hour format (e.g., 1530 for 3:30 PM)
    int arrivalTime;    // Arrival time in 24-hour format (e.g., 1830 for 6:30 PM)
    int price;          // Price of the flight
};

// Function to swap two flights
void swapFlights(Flight &a, Flight &b) {
    Flight temp = a;
    a = b;
    b = temp;
}

// Function to sort flights by departure times
void sortByDepartureTime(Flight flights[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (flights[j].departureTime > flights[j+1].departureTime) {
                swapFlights(flights[j], flights[j+1]);
            }
        }
    }
}

// Function to (bubble)sort flights by arrival times
void sortByArrivalTime(Flight flights[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (flights[j].arrivalTime > flights[j+1].arrivalTime) {
                swapFlights(flights[j], flights[j+1]);
            }
        }
    }
}

// Function to sort flights by prices
void sortByPrice(Flight flights[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (flights[j].price > flights[j+1].price) {
                swapFlights(flights[j], flights[j+1]);
            }
        }
    }
}

// Function to print the list of flights
void printFlights(Flight flights[], int n) {
    for (int i = 0; i < n; i++) {
        cout << "Flight ID: " << flights[i].flightID
             << ", Departure: " << flights[i].departureTime
             << ", Arrival: " << flights[i].arrivalTime
             << ", Price: $" << flights[i].price << endl;
    }
```

```
}

int main() {
    // Array of flights
    Flight flights[] = {
        {101, 1530, 1830, 400},
        {102, 1230, 1430, 350},
        {103, 1600, 2000, 450},
        {104, 1330, 1530, 300}
    };

    int n = sizeof(flights) / sizeof(flights[0]);

    // Sorting flights by departure time
    cout << "Flights sorted by Departure Time:" << endl;
    sortByDepartureTime(flights, n);
    printFlights(flights, n);
    cout << endl;

    // Sorting flights by arrival time
    cout << "Flights sorted by Arrival Time:" << endl;
    sortByArrivalTime(flights, n);
    printFlights(flights, n);
    cout << endl;

    // Sorting flights by price
    cout << "Flights sorted by Price:" << endl;
    sortByPrice(flights, n);
    printFlights(flights, n);

    return 0;
}
```

Explanation:

1. **Structure `Flight`**:
   - Contains four integer fields: `flightID`, `departureTime`, `arrivalTime`, and `price`.
   - This structure will represent each flight with a simple integer-based approach (no strings or vectors used).

2. **`swapFlights` Function**:
   - A helper function that swaps two `Flight` structures. It's used by the sorting functions to exchange flight details.

3. **Sorting Functions**:
   - **`sortByDepartureTime`**: Sorts flights by their departure time in ascending order.
   - **`sortByArrivalTime`**: Sorts flights by their arrival time in ascending order.
   - **`sortByPrice`**: Sorts flights by their price in ascending order.
   - All three functions use the basic Bubble Sort algorithm.

4. **`printFlights` Function**:
   - Prints the list of flights in a readable format.

5. **Main Function**:
   - Defines an array of flights with sample data.
   - Calls each sorting function (`sortByDepartureTime`, `sortByArrivalTime`, and `sortByPrice`) and prints the sorted list of flights.

Using linked list

```
#include <iostream>
using namespace std;

// Structure for Flight node in the linked list
struct Flight {
    int flightID;
    int departureTime;
    int arrivalTime;
    int price;
    Flight* next;
};

// Function to insert a new flight node at the end of the list
void appendFlight(Flight** head_ref, int id, int departure, int arrival, int price) {
    Flight* new_flight = new Flight();
    new_flight->flightID = id;
    new_flight->departureTime = departure;
    new_flight->arrivalTime = arrival;
    new_flight->price = price;
    new_flight->next = NULL;

    if (*head_ref == NULL) {
        *head_ref = new_flight;
    } else {
        Flight* temp = *head_ref;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = new_flight;
    }
}

// Swap two flight nodes (by swapping their data)
void swapFlights(Flight* a, Flight* b) {
    int tempID = a->flightID;
    int tempDep = a->departureTime;
    int tempArr = a->arrivalTime;
    int tempPrice = a->price;

    a->flightID = b->flightID;
    a->departureTime = b->departureTime;
    a->arrivalTime = b->arrivalTime;
    a->price = b->price;

    b->flightID = tempID;
    b->departureTime = tempDep;
    b->arrivalTime = tempArr;
    b->price = tempPrice;
}

// Sort the linked list by departure time
void sortByDeparture(Flight* head) {
```

```
    Flight* i;
    Flight* j;
    for (i = head; i != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->departureTime > j->departureTime) {
                swapFlights(i, j);
            }
        }
    }
}

// Sort the linked list by arrival time
void sortByArrival(Flight* head) {
    Flight* i;
    Flight* j;
    for (i = head; i != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->arrivalTime > j->arrivalTime) {
                swapFlights(i, j);
            }
        }
    }
}

// Sort the linked list by price
void sortByPrice(Flight* head) {
    Flight* i;
    Flight* j;
    for (i = head; i != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->price > j->price) {
                swapFlights(i, j);
            }
        }
    }
}

// Function to print the flight list
void printFlights(Flight* head) {
    Flight* temp = head;
    while (temp != NULL) {
        cout << "FlightID: " << temp->flightID << ", Departure: " << temp->departureTime
             << ", Arrival: " << temp->arrivalTime << ", Price: " << temp->price << endl;
        temp = temp->next;
    }
}

int main() {
    Flight* head = NULL;

    // Adding flights to the list
    appendFlight(&head, 101, 1200, 1400, 300);
    appendFlight(&head, 102, 900, 1100, 200);
    appendFlight(&head, 103, 1500, 1700, 400);
    appendFlight(&head, 104, 1000, 1300, 250);
```

```
    cout << "Flights sorted by Departure Time:\n";
    sortByDeparture(head);
    printFlights(head);

    cout << "\nFlights sorted by Arrival Time:\n";
    sortByArrival(head);
    printFlights(head);

    cout << "\nFlights sorted by Price:\n";
    sortByPrice(head);
    printFlights(head);

    return 0;
}
```